Deep Learning for Biological Discovery Across Omics Layers: From Phenomics to Agentic Reasoning

Dissertation Proposal

The University of Tennessee, Knoxville

Peter Kruse

June 2026

Contents

List of Tables

List of Figures

Executive Summary

0.1 Motivation

Modern biology has become increasingly data-rich but interpretation-limited. Data collection is now cost-effective in many realms, enabling the construction of novel datasets. High-throughput imaging technology has led to population-scale phenotyping datasets, while advanced sequencing methods allow researchers to capture genome-scale variation across populations. Global consortia efforts have standardized environmental metadata across climates and sites, while novel experimental data and computational frameworks have prompted the curation and creation of biological networks and large literature corpora. These data span fundamentally different structures: images contain spatial, pixel-level information; genotypes comprise high-dimensional, sparse signals; environments exist in a continuous and contextual space; and networks represent relational graph structures. Handling these different structures requires systems with different inductive biases; no single model class can handle them all.

0.2 Thesis Statement

While progress has been made applying deep learning to localized areas, biological data are heterogeneous and structurally varied, making the gains from current deep learning methods unequally distributed. Because of the nature of biological data,

bridging the gap between inputs and interpretation requires problem-specific systems. Architectures, objectives, and benchmarks must be designed with appropriate inductive biases. This proposal demonstrates the utility of this framework across three linked stages of biological discovery: *measurement*, *prediction*, and *interpretation.*

0.3 Chapter 2 Summary

Chapter 2 addresses the *measurement* stage by extracting quantitative phenotypes from raw imagery. No deep learning framework currently exists for phenotyping pennycress (*Thlaspi arvense L.*), an emerging oilseed cover crop. In this work, we present an automated deep learning pipeline for intensive pennycress seed pod phenotyping, extracting over 50 phenotypes per image. We implemented U-Net, a state-of-the-art deep learning model, to semantically segment wing, envelope, and seed pixels in seed pod images, and we created an open-source ground truth dataset of 365 hand-annotated label images for training and validation. Our model achieved over 96% accuracy for wing segmentation, 95% accuracy for envelope segmentation, and almost 80% accuracy for seed segmentation. We further validated the model through a population-scale study on over 11,000 seed pods from 771 unique genotypes.

0.4 Chapter 3 Summary

Chapter 3 addresses the *prediction* stage by mapping genotype and environment to phenotype. Accurate maize yield prediction is crucial for global food security, but remains challenging due to complex genotype-by-environment (G×E) interactions. In this work, we present a unified transformer architecture for maize yield prediction that tokenizes single-nucleotide polymorphism (SNP) markers and environmental covariates into a single input sequence, learning G×E interactions through self-attention. Mixture-of-experts (MoE) layers with Top-K routing replace the dense feed-forward blocks, allowing the model to specialize across genetic and environmental

contexts. We benchmark this architecture against a suite of variants, including a three-prong encoder-fusion design with dedicated genotype, LD, and environment branches. Training on the publicly available Genomes2Fields (G2F) prediction competition dataset comprising 21 environments over 11 years, we achieve a within-environment Pearson correlation coefficient (PCC) of 0.423 on the 2024 test set, approaching the competition-winning score of 0.45 and exceeding the strongest previously published deep learning baselines we evaluated.

0.5 Chapter 4 Summary

Chapter 4 addresses the *interpretation* stage by reasoning over biological networks to produce evidence-grounded mechanistic hypotheses. While expert-curated corpora and AI-based methods have produced rich relational structures at scale, downstream synthesis remains constrained by human bandwidth and model cost. We propose MENTOR-RL, an open-source, tool-using reasoning agent trained with reinforcement learning over verifiable biological-network tasks. MENTOR-RL consists of a single policy operating in an act-verify agent loop and is trained with deterministic, schema-grounded rewards that require no proprietary judge models. We expect MENTOR-RL to achieve a composite F_1 score competitive with frontier models at a substantially lower per-query inference cost, while producing human-readable interpretations with preserved provenance.

0.6 Completion Plan and Expected Outputs

Chapters 2 and 3 are in preparation for submission to [venue] and [venue], respectively; Chapter 4 is proposed work targeting [venue/timeline]. Models in Chapters 3 and 4 are trained at scale using distributed training on the OLCF Frontier supercomputer [4]. Upon publication, we will release the open-source ground truth dataset of hand-annotated seed pod images and segmentation pipeline (Chapter **??**); the training and

evaluation code (Chapter **??**); and the training environment, evaluation harness, and trained checkpoints across pipeline stages (Chapter **??**).

Chapter 1

Literature Review

1.1 Deep Learning History

According to Goodfellow et al., the history of neural network research can be divided into three eras: cybernetics, connectionism, and deep learning [38]. In the following sections, we adopt this organizational structure to explore the development of deep learning.

1.1.1 Cybernetics

The cybernetics era, spanning the 1940s and 1960s, was characterized by early efforts to formalize neural computation through the mathematical modeling of biological nervous systems. The first foundational contribution from this period was presented by McCulloch and Pitts in *A Logical Calculus of Ideas Immanent in Nervous Activity* [79]. The authors provided a modeling framework for neuronal activity through the lens of formal logic; each neuron was framed as a binary computational unit within a larger symbolic logic network. Neuronal activation was determined by inputs corresponding to excitation and inhibition; an output was produced when the aggregated inputs exceeded a fixed threshold. Neurons emitted discrete logical outputs $True$ or $False$, enabling the construction of networks that implemented

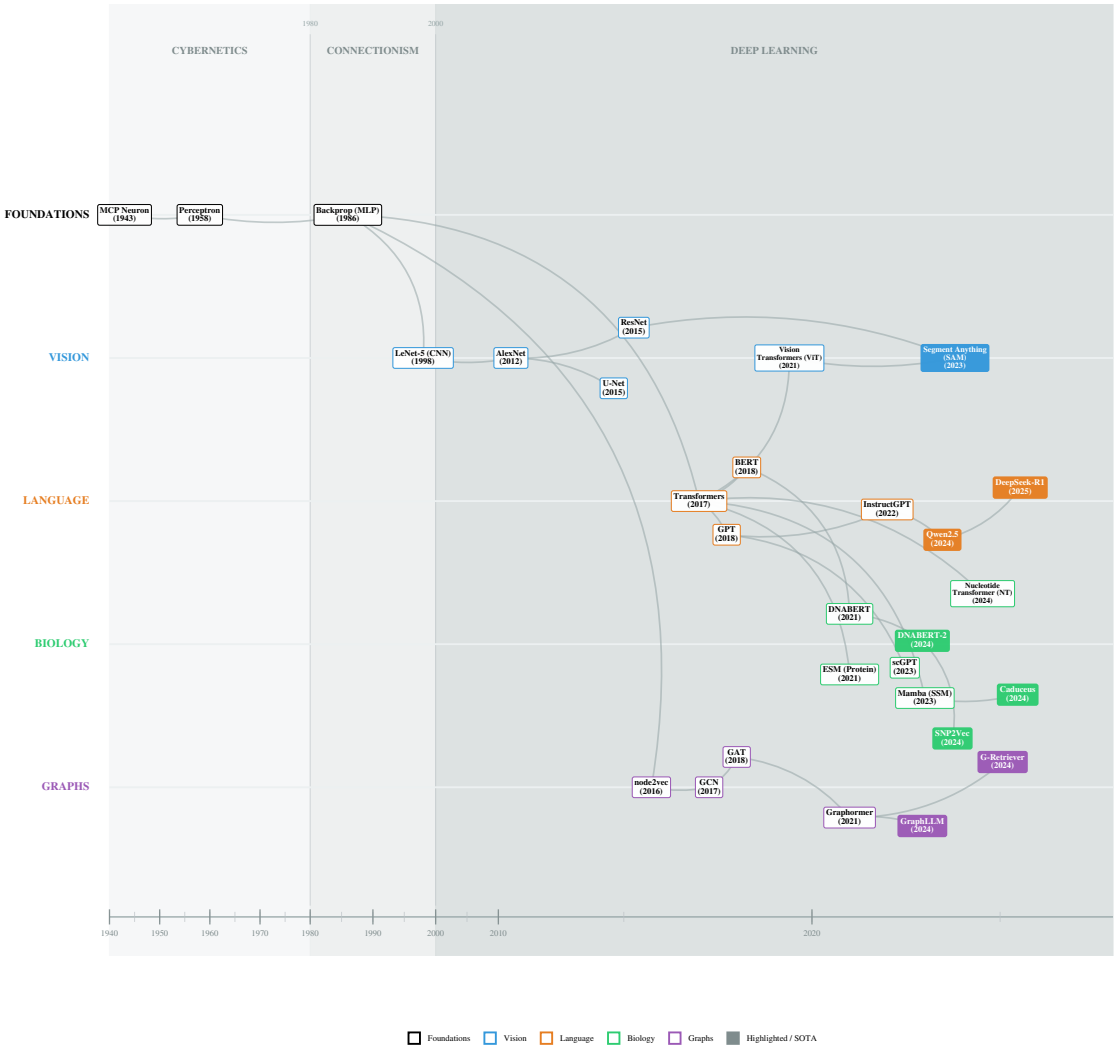


Figure 1.1: History of Deep Learning (1940-2026)

logical functions. Through this connected framework, McCulloch and Pitts aimed to model neural circuitry in the brain. Importantly, they emphasized that these networks worked over discrete time steps t, allowing them to represent sequential dynamics and foreshadowing later concepts like recurrence and memory in neural systems.

Despite establishing a foundational framework for neural computation, the McCulloch-Pitts model exhibited several fundamental limitations. Most notably, the formulation lacked a learning mechanism: network structure and parameters were fixed, requiring manual design rather than learning from data. This static framework stood in contrast to Donald Hebb's work on synaptic plasticity, which postulated that neural pathways are strengthened by repeated co-activation [45]. Without such a mechanism, the behavior of the McCulloch-Pitts networks could not be modified in response to novel stimuli, limiting this framework's utility for modeling adaptive biological or cognitive processes.

Furthermore, relying on a purely logical formulation imposed strict representational constraints. Neurons were restricted to binary activation states, resulting in rigid decision boundaries and precluding the learning of continuous representations. Together, these limitations prevented the McCulloch-Pitts networks from scaling to tasks requiring robust function approximation and learning from noisy, high-dimensional data.

The limitations of the McCulloch-Pitts neuron motivated subsequent efforts to develop neural models capable of adaptation, most notably Frank Rosenblatt's 1958 work *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain* [105]. Rosenblatt described the perceptron as "a hypothetical nervous system," reflecting an effort to maintain biological inspiration while introducing mechanisms for experience-driven learning. In contrast to the symbolic logic-based formulation of McCulloch and Pitts, Rosenblatt adopted a probabilistic perspective. He argued that the extensive assumptions and structure imposed by their model limited its ability to yield meaningful biological insight.

Instead, the perceptron implemented a paradigm that treated data as statistically separable, with a learning mechanism that allowed connections to be reinforced or discouraged via positive or negative stimuli from the data.

While innovations introduced by the perceptron, such as trainable neurons and learning from data, are central to modern deep learning models, the original model had significant shortcomings. Most notably, Rosenblatt's formulation was incapable of representing non-linearly separable functions, with the XOR problem serving as a canonical example. This limitation was a consequence of the perceptron's reliance on linear decision boundaries and its restriction to single-layer architectures, and it severely limited its expressivity. Additionally, Rosenblatt's training rules could not be extended to multi-layer networks due to the lack of a credit assignment mechanism across multiple layers of computation. Combined with the use of binary outputs, these constraints severely limited the functions and representations that could be learned.

These limitations were rigorously analyzed by Minsky and Papert in their 1969 book *Perceptrons*, whose formal results played a significant role in altering public perception of AI research and contributed to a temporary decline in funding.

1.1.2 Connectionism

Although Minsky and Papert's work provided a rigorous exploration of the limitations of single-layer perceptrons, their critique did not explore whether such constraints could be overcome by multi-layer networks equipped with a credit assignment mechanism. The subsequent era of neural network research, commonly referred to as *connectionism*, was defined by the efforts to address this challenge through the development of multi-layer architectures and the backpropagation algorithm. During this period neural networks also transitioned from primarily symbolic and biologically motivated abstractions toward increasingly general-purpose learning systems. As learning algorithms became more complex and models increased in scale, research

increasingly began to favor mathematical tractability over biological interpretability [107, 38].

In 1986, Rumelhart, Hinton, and Williams published *Learning Representations by Back-Propagating Errors* [107], demonstrating that multi-layer neural networks could be effectively trained through an general solution to the credit assignment problem. Although the mathematical foundations for efficiently computing gradients, known as reverse-mode automatic differentiation, had been previously derived by Seppo Linnainmaa [74] and applied to neural networks by Paul Werbos [138], Rumelhart, Hinton, and Williams popularized the method within the AI community.

The backpropagation algorithm is a gradient-based learning procedure that propagates error information from the network's output backward through successive layers, incrementally modifying each parameter. Using networks trained with backpropagation, the authors showed that non-linearly separable functions could be learned by multilayer networks without relying on hand-designed learning rules.

Backpropagation relies on the differentiability of both the loss function and the network's activation functions, which allow gradients with respect to all parameters to be computed efficiently via the chain rule. This formulation enabled learning signals to be propagated to all network layers, in contrast to earlier approaches like the perceptron, which required a manually specified learning procedure. The formulation of weight updates as an optimization problem allowed neural network training to be extended to more complex multilayer architectures. In addition to backpropagation, this work made several other contributions, such as an explicit learning rate, momentum-based updates, and gradient accumulation, all of which are core to modern neural network optimization.

Beyond their algorithmic contribution, Rumelhart, Hinton, and Williams provided a conceptual framework for understanding multilayer networks. They characterized networks as self-organizing systems that were capable of learning internal representations from data, positing that hidden units in multilayer networks encode features of increasing complexity as network depth increases. These concepts formed the basis

of future work in representation learning and motivated the exploration of deeper network architectures to model more complex structures in data.

The representational capabilities of neural networks were mathematically substantiated in 1989, when Cybenko and Hornik et al. independently published proofs of the Universal Approximation Theorem. These works demonstrated that a feed-forward network with a single hidden layer and a finite number of neurons can approximate any continuous function on a compact set [48, 20]. While Cybenko's work was restricted to networks utilizing sigmoidal activation functions, Hornik et al. extended this result, proving that universality holds for any bounded, non-constant, and monotonically increasing activation function. This work provided a definitive rebuttal to Minsky and Papert's critique of perceptrons [82], confirming that the limitations they identified were specific to single-layer architectures.

Despite these contributions, connectionist-era neural networks exhibited several limitations relative to modern deep learning models. Most notably, training was restricted by computational resources available at the time, which limited model size and large-scale experimentation. Neural networks were trained on general-purpose central processing units (CPUs), making it impractical to explore deeper architectures or large datasets.

In addition to computational constraints, optimization challenges further limited the effectiveness of early multilayer networks. The authors noted that the back-propagation algorithm could fail in the presence of local minima. With too small a learning rate, weight updates are not enough to traverse the loss landscape toward the global minimum. Furthermore, due to the use of activations such as sigmoid and tanh, gradients could become saturated during backpropagation. Excessively large or small values would fall near the asymptote of these functions, leading to vanishingly small values. As such, the resulting gradients would be small, multiplicatively reducing the learning signal in successive layers. These limitations prevented networks from increasing in depth and learning more complex representations, prompting the development of new methods to overcome these challenges.

1.1.3 Deep Learning

The transition from connectionism to deep learning emerged from convergent improvements in data, hardware, and algorithms during the late 1990s and early 2000s. These innovations addressed fundamental challenges of the connectionist era, such as vanishing gradients, computational bottlenecks, and convergence failures. Consequently, training networks with more layers became feasible and effective, leading to the era of deep learning, a term popularized by Geoffrey Hinton in 2006 [46]. The pursuit of depth was built on the theoretical foundations laid by Rumelhart, Hinton, and Williams, who demonstrated that hidden layers allow networks to learn more complex hierarchical representations. [107, 68].

A key innovation in the deep learning era was the shift from CPUs to Graphics Processing Units (GPUs). The physical architecture of the GPU is well-suited for homogeneous, parallelizable, and batched operations, specifically massive matrix multiplications central to neural network training. In 2009, Raina et al. published *Large-Scale Deep Unsupervised Learning Using Graphics Processors*, in which they trained Deep Belief Networks (DBNs) [46] and sparse coding networks [88] using GPUs [99]. By storing parameters in the GPU's global memory, parallelizing homogeneous thread operations in stream processors, and using efficient scheduling to reduce overhead times, the authors were able to achieve up to a 72x speedup in DBN training and a 15x speedup in sparse coding training. This acceleration rendered previously intractable problems computationally feasible; the reduction in training time enabled the exploration of deeper architectures and larger datasets, expanding the frontier of neural network training.

In order to fully leverage the benefits of GPU acceleration for neural network training, large, annotated datasets to were necessary to avoid overfitting and improve generalization. The earliest advances in dataset construction occurred in the image domain; in the early 2000s, the MNIST dataset of handwritten digits [71] served as the benchmark for neural network training. However, this dataset comprised

only 10 image classes within a narrow domain, meaning that models trained on this data failed to capture general image recognition capabilities. To overcome these limitations, datasets such as Caltech101 [27], PASCAL VOC [26], and MSRC [116] were developed, providing a wider variety of images and classes for training general-purpose computer vision models. Despite containing more subject diversity than MNIST, these datasets were still relatively small and lacked intra-class variability, with images often being standardized and lacking occlusions, translations, and other perturbations. In 2009, a breakthrough was made with the development and release of the ImageNet dataset [23]. The authors of ImageNet leveraged the increasing amount of data available on search engines and the advent of crowdsourcing platforms like Amazon Mechanical Turk to produce a dataset of 3.2 million annotated images following the hierarchical structure of WordNet [28]. The availability of large-scale datasets such as ImageNet encouraged the development of neural networks with more layers to capture increasingly abstract and general image representations. This paradigm shift was further catalyzed by the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [108], which will be discussed further in Section **??**.

As hardware capabilities improved and massive datasets became available, several algorithmic innovations enabled researchers to train deeper, larger-scale models. One notable breakthrough was the rectified linear unit, or ReLU, which supplanted the sigmoid and hyperbolic tangent (tanh) activation functions that were previously standard [107, 70]. Although ReLU was first utilized in Fukushima's *Cognitron* and *Neocognitron* architectures [29, 30], it became widely adopted only after Glorot et al. published *Deep Sparse Rectifier Networks* in 2011. Glorot et al. showed that ReLU was not only more biologically plausible than its counterparts but also empirically outperformed them in both computer vision and sentiment analysis, especially with deep architectures. ReLU activation enforced network sparsity by setting negative activations to zero and mitigated the vanishing gradient problem with strictly linear positive activations, preventing gradient saturation.

To fully leverage these properties, weight initialization techniques became widely adopted in deep networks. Building on the work of Glorot and Bengio, who introduced 'Xavier initialization' to control the activation variance in sigmoidal networks [36], He et al. introduced 'He initialization' to account for the rectification nonlinearity of ReLU [43]. These innovations facilitated the training of significantly deeper networks, cementing ReLU as a foundational component of the deep learning era.

To address the generalization challenges posed by these high-capacity models, Srivastava et al. introduced dropout in 2014 [120]. Although large datasets such as ImageNet were becoming available, many domains lacked access to millions of labeled training examples. Consequently, deep networks tended to overfit smaller datasets, fitting noise and exhibiting high variance. Dropout ameliorated this issue by regularizing the co-adaptations between neurons. Biologically inspired by sexual reproduction, which reduces complex co-adaptation to make genes more robust, dropout probabilistically removes neurons from the network during training. As a result, neurons in the network cannot co-adapt with each other and must instead learn to function with a randomly chosen sample of units. As the authors stated, "Complex co-adaptations can be trained to work well on a training set, but on novel test data they are far more likely to fail than multiple simpler co-adaptations that achieve the same thing." The utility of dropout is two-fold: it can enforce sparsity and considerably improves generalization error. Due to these factors, dropout has been ubiquitous in state-of-the-art architectures throughout the deep learning era.

In addition to ReLU and dropout, Ioffe and Szegedy introduced batch normalization in 2015, which allowed networks to be further extended [53]. The authors aimed to remedy internal covariate shift a phenomenon where the distribution of activations in hidden layers shifts continuously based on parameter updates in previous layers. To mitigate this, the authors proposed normalizing layer activations around the mini-batch mean and variance. They demonstrated that batch normalization accelerates training and acts as a regularizer while also achieving state-of-the-art performance across multiple deep learning benchmarks. While Santurkar et al. argued in 2018

that batch normalization helps optimization through smoothing the optimization landscape instead of reducing internal covariate shift [109], the technique remains a standard component in modern deep neural networks, facilitating the learning of increasingly complex representations.

Along with the architectural changes that made deep learning possible, the Adaptive Moment Estimation (Adam) optimizer was introduced to make weight updates for deeper networks more efficient [60]. Adam utilizes the first and second moments of the gradients to compute adaptive learning rates for each weight with little additional computation. The authors liken the resulting update rule to a "trust region" method: because the effective step size is bounded by the learning rate, optimization is effectively constrained to a neighborhood where the gradient estimate remains reliable. This promotes stability when gradients are sparse or noisy, allowing for faster convergence. Consequently, Adam has become the primary choice of optimizer in modern deep learning models.

The deep learning era was marked by the confluence of hardware improvements, data availability, and algorithmic innovation. These three factors enabled researchers to explore deep architectures that were previously impossible to train. As depth increased, a major paradigm shift occurred: the transition from *feature engineering* to *feature learning* [68]. While prior approaches relied on hand-crafted features to structure data, deeper networks allowed for complex representations to be learned directly in latent space, deriving hierarchical features with backpropagation instead of by hand.

With this shift away from manual feature engineering, research focus turned toward architectural and algorithmic design, with inductive biases tailored to specific data modalities. Computer vision assumed translation invariance, language models assumed sequentiality and contextual dependence, and graph deep learning models incorporated permutation invariance. These inductive biases align closely with the properties of biological data, making the history and current state of deep learning across modalities crucial for applications in phenomics, genomics, and systems biology.

The remainder of this review is organized as follows: we cover computer vision in Section **??**, language modeling in Section **??**, and graph learning in Section **??**. Additionally, we will explore techniques to scale neural networks to massive modern datasets in Section **??**. We will conclude with a review of how these techniques are applied to various biological subdomains in Section **??**.

1.2 Computer Vision

Computer vision served as the primary proving ground for neural network techniques and architectures during the deep learning era. This domain was the first where deep learning achieved widespread consumer adoption [71] and later surpassed human performance [43]. Breaking these performance plateaus shifted the perception of deep learning from an esoteric field into a rapidly evolving discipline with disruptive applications. This success was not accidental; rather, the alignment between the structure of image data and the inductive biases of neural architectures, specifically convolutional neural networks (CNNs), allowed deep learning models to excel at learning complex representations. By integrating translation invariance, local connectivity, and hierarchy into neural architectures, researchers created biologically inspired vision models, grounded in Hubel and Wiesel's model of the mammalian visual cortex [50, 51], validating the connection between biological and artificial vision.

Several factors catalyzed rapid advancement of deep learning. The first was the establishment of standardized benchmarks that promoted competition and innovation in the computer vision community. The PASCAL VOC challenge [26], and subsequently the ILSVRC [108], provided annual venues to evaluate visual recognition algorithms. These datasets, distinguished by their massive scale and public availability, provided a robust stage to validate methods while fostering competition and connectivity. This created a feedback loop that encouraged research participation and innovation in computer vision, leading to rapidly dropping competition error rates (see Figure **??**). Due to the challenging nature of the ILSVRC

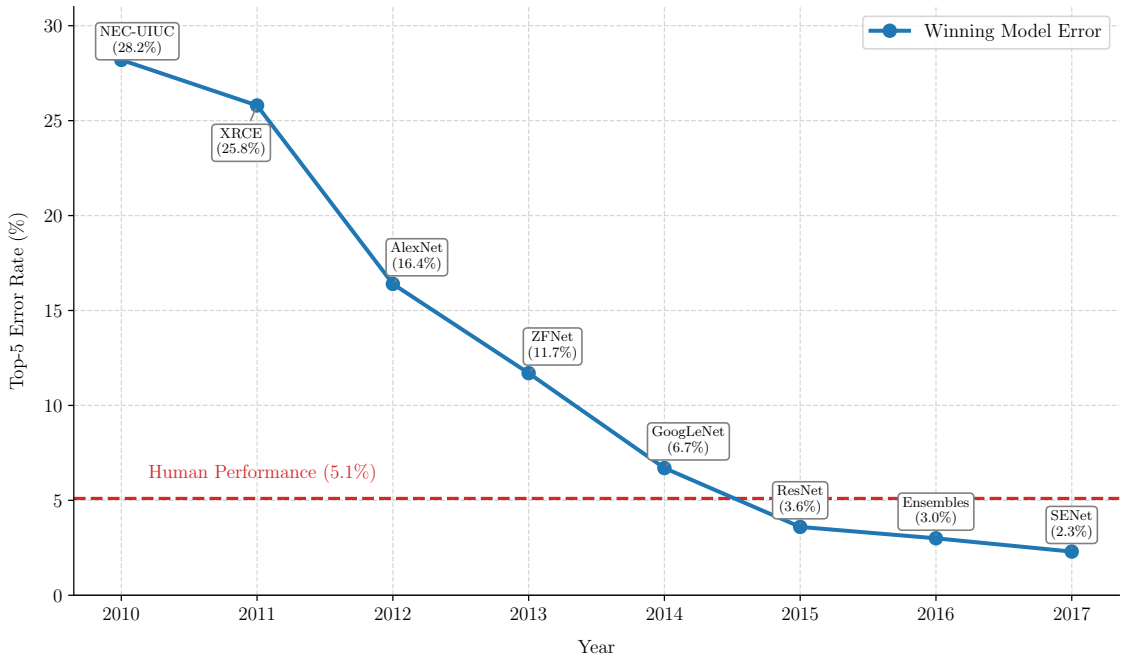


Figure 1.2: ILSVRC Top-5 Error Rate by Year (2010-2017)

and its popularity, the competition also served as a "test bed," where techniques that succeeded in the image domain became part of the standard toolkit for other deep learning subdomains, including biological modeling. Foundational methods widely adopted after their demonstration in the ILSVRC include GPU parallelism [66], residual connections [41], and data augmentation techniques [66].

The confluence of these factors enabled a transition from foundational benchmarking to biologically significant innovation. As performance on general image classification saturated, focus shifted to higher-order tasks, such as object detection, semantic segmentation, and automated feature extraction. These methodologies proved directly applicable to spatial and visual biological data, catalyzing innovation in image-based phenotyping, geospatial modeling, and multimodal artificial intelligence. Consequently, examining the evolution of computer vision is essential to understanding the technological development of modern biological modeling.

1.2.1 Origins

Much like general deep learning research (Section **??**), computer vision research began as a biologically inspired field grounded in neuroscience principles. In the late 1950s and early 1960s, Hubel and Wiesel provided the theoretical underpinnings for modern architectures with their work on the mammalian visual cortex [50, 51]. They showed that vision is a hierarchical process built on detecting and combining local features. The visual cortex does not respond to the retinal input holistically; instead, neurons respond to stimuli within small, specific regions called receptive fields. Additionally, Hubel and Wiesel identified two types of cells involved in processing this visual information: simple cells and complex cells. While simple cells fire only when a specific feature, like a horizontal or vertical line, appears in a precise location, complex cells activate when the feature appears anywhere in the receptive field. Based on these fundamental units, they proposed a hierarchical model of visual processing: simple cells connect to complex cells, which connect to hypercomplex cells, allowing the visual cortex to build a layered representation that abstracts from lines to shapes and eventually to objects.

Kunihiko Fukushima built on this work by introducing the cognitron in 1975 and the neocognitron in 1980 [29, 30]. A precursor to modern CNNs, the neocognitron operationalized the concepts of receptive fields, simple cells, and complex cells from Hubel and Wiesel. Instead of having fully connected layers like the perceptron [105], the neocognitron featured a shared weight mechanism. This simulated simple cells (referred to as "S-cells" by Fukushima) by applying the same filter to each part of the image, meaning a cell would activate any time its filter matched the input pattern in a region. Multiple S-cells could be applied to the image in parallel to detect multiple features. These features were then propagated to complex cells, or C-cells, for transformation-invariant feature detection. S-cells are equivalent to convolution operations, and C-cells are equivalent to pooling operations. The combination of multiple S-C layers resulted in a similar hierarchical structure to the model of

visual processing introduced by Hubel and Wiesel [50, 51]. Although Fukushima made foundational contributions to CNN architecture, the neocognitron relied on unsupervised competitive learning rather than backpropagation, which hindered its standalone success for complex supervised tasks.

In 1989, Yann LeCun combined the architectural foundations of Fukushima [29, 30] with the learning rules of Rumelhart et al. [107] to train the first modern CNNs [72]. LeCun used backpropagation to train a CNN for recognizing digits from the United States Postal Service. The model achieved a 1% error rate and 9% rejection rate, demonstrating state-of-the-art performance on a real-world task. This result showed that utilizing convolutions for computer vision tasks was a promising approach; because the model learned hierarchical features, the need for manual preprocessing was minimized. Furthermore, the computational burden of these networks was lower than fully-connected networks due to their shared-weight design. Additionally, LeCun argued that the reduction in free parameters caused by the architectural choice reduced overfitting, providing more evidence for the potential of CNNs in computer vision.

LeCun produced multiple iterations on this approach, introducing LeNet-5 in 1998 [71]. This work expanded the results from LeCun's original paper by scaling the dataset and model size while providing a comparison to other gradient-based models. LeNet-5 achieved performance superior to most existing methods and comparable to Support Vector Machines (SVMs), while offering significantly faster inference speeds. LeCun noted that "better pattern recognition systems can be built by relying more on automatic learning and less on hand-designed heuristics" [71]. The combination of scale and performance in these experiments solidified CNNs as one of the primary methods for image classification. Additionally, the LeNet-5 paper introduced a modular paradigm for neural network programming, introducing various forward and backward propagation functions that could be composed while still storing a computational graph. This framework provided a predecessor to popular deep learning libraries such as Caffe [58], TensorFlow [78], Pytorch [92], and Jax [11].

Despite CNNs showing successful performance at scale, training was still constrained by hardware and data limitations. LeNet-5 took three days to train for 20 epochs, and the MNIST dataset was still small by modern standards, containing only around 60,000 images. However, as discussed in Section **??**, innovations in these realms promoted more successful computer vision research, with the ILSVRC [108] serving as the catalyst for rapid progress.

1.2.2 The ImageNet Challenge

As introduced in Section **??**, the ImageNet dataset [23] revolutionized computer vision research, providing a massive, open-source dataset for researchers to benchmark their methods. Due to its popularity, the creators of ImageNet introduced the ILSVRC, which created a standardized benchmark for the deep learning community. As noted earlier, many seminal contributions used the ILSVRC as their primary benchmarking dataset. We will discuss the impact of key ILSVRC participants in this section.

The first major breakthrough related to the ILSVRC was the introduction of AlexNet [66], which won the competition in 2012. AlexNet achieved a reduction in top-5 error rate of over 10 percentage points compard to the previous winner, producing "by far the best results ever reported on these datasets" at the time [66]. The AlexNet architecture revolutionized deep learning with several contributions. Primarily, the authors were the first to utilize model parallelism with GPUs to train larger models more efficiently. They used two GPUs, partitioning convolutional filters between the two to parallelize processing. Despite being one of the largest CNNs ever at the time, training took a relatively modest five to six days. Furthermore, AlexNet was the first network to utilize dropout [120], ReLU [37], and data augmentation at scale. The adoption of these techniques, combined with the competition-winning performance they yielded, solidified AlexNet as a foundational work and exemplified the superiority of deep learning for image recognition over other techniques, like SVMs, which were popular at the time. It is also important to note that AlexNet's

success was a precursor to modern deep learning scaling laws, which will be discussed further in Section **??**; the authors demonstrated that using more compute and data had a direct relationship with performance, a trend that continued to be explored in the following years across various deep learning subdomains.

In 2014, two innovative architectures achieved first and second place in the ILSVRC. GoogLeNet [123], developed by researchers at Google, won the competition, while VGGNet took second place. GoogLeNet utilized an architecture called "Inception," which enabled the training of even deeper CNNs while using fewer parameters. Each inception block balanced receptive field focus by using 1×1, 3×3, and 5×5 convolutional kernels in parallel in a single layer; the varied kernel sizes created an implicit balance between hierarchy and localization. Crucially, to reduce the number of parameters in the network, and thus overfitting, GoogLeNet utilized 1 global average pooling to transform the inception outputs for classification rather than the dense layers used in prior work. Due to the shared weights of convolutions, this greatly reduced the parameter burden compared to using fully-connected layers, which previous architectures like AlexNet had relied on for reshaping outputs [66]. Finally, GoogLeNet leveraged auxiliary classification to increase network depth. The authors added outputs at intermediate layers in the network; by performing backpropagation from these auxiliary objectives, they ensured a strong gradient signal throughout the network, even with increasing depth. The combination of these innovations produced the most performant model of the time.

While GoogLeNet's Inception architecture showed that a complex, highly engineered CNN could yield state-of-the-art performance, VGGNet took an alternate approach, demonstrating that a homogeneous network with many layers of small convolutions could yield comparable performance [117]. The authors tested CNNs of various depth while exclusively using 3×3 convolutional filters, arguing that stacks of smaller filters were equivalent to a single, larger filter while fitting a more discriminative function with fewer parameters. They demonstrated this result empirically, achieving second place in the ILSVRC and first place in the ImageNet

localization competition. This homogeneous, stacked architecture provided an easily scalable structure, and its simplicity led to its adoption across multiple other subdomains.

In 2015, superhuman performance was formally achieved in the ILSVRC with the introduction of ResNet [41]. The primary focus of the ResNet architecture was combating the vanishing gradient problem, which, despite being partially addressed by innovations such as ReLU [37], was still prevalent as network depth exceeded 30 layers. The ResNet team introduced residual learning, where instead of fitting a function, $h(x)$, its residual, $h(x) - x = f(x)$, was learned. Consequently, layer mappings could be reformulated as learning $f(x) + x$, including a direct connection from the input to the output of a layer. This directly addressed gradient saturation by providing a residual stream without exposure to a nonlinearity, stabilizing gradient flow. By adding residual connections to an architecture inspired by VGGNet, the authors were able to train networks with a depth of up to 152 layers. The innovations introduced in ResNet have been adopted by numerous state-of-the-art architectures across subdomains, showing the impact of this work.

Over its life cycle from a difficult, open research competition to a saturated benchmark, the ILSVRC yielded many innovative approaches to image classification. Model parallelism [65], dropout [120], ReLU [37], homogeneous block architectures [117], and residual connections [41] are all used in various architectures across subdomains today. In the next section, we will discuss how these methods were incorporated to address the challenges of semantic segmentation and object detection.

1.2.3 Semantic Segmentation

As performance on the ILSVRC improved, the benchmark became saturated; state-of-the-art models reached an accuracy plateau where meaningful improvements became diminishingly marginal. Due to this saturation, focus shifted to more complex tasks such as semantic segmentation, where specificity and localization

carried greater weight. In biology, semantic segmentation is critical; for example, pixel-level classification enables researchers to extract intensive phenotypes from images, facilitating population-scale studies. Therefore, it is important to understand advances in semantic segmentation to understand how best to apply it to biological data.

While semantic segmentation was historically strongly tied to the ImageNet dataset, one of the first state-of-the-art segmentation models was developed for biomedical image segmentation. U-Net, developed in 2015, is a fully convolutional architecture that produces pixel level classifications to create image segmentation maps. The architecture features a contractive path to capture hierarchical and contextual features followed by a symmetric expansive path to provide specific localizations within the image. The contractive and expansive paths are also linked via skip connections. Functionally similar to residual connections [41], skip connections improve gradient flow in the model while also enabling the combination of hierarchical features with localized features via concatenation in the expansive feature map [104]. U-Net achieved state-of-the-art performance on both *Drosophila* nerve membrane segmentation and cellular segmentation, solidifying its role as a biologically useful deep learning architecture.

Despite the success of U-Net, its supervised nature meant that ground-truth pixel-level segmentation data was necessary for training. Such data is expensive and tedious to collect, so naturally, researchers began to explore other architectures to reduce the burden of collecting hand-labeled segmentations. One of the first attempts to explore deep, unsupervised image segmentation was W-Net [142]. This architecture combined two U-Nets to produce a segmentation map without supervision. The first U-Net produced a k-channel representation of the image, with k representing the number of classes for segmentation. This segmentation map was then propagated through another U-Net, which reconstructed the input. The intermediate segmentation map was evaluated with a soft normalized cut loss function, which maximizes association between pixel groups, while the final reconstruction was evaluated with

a reconstruction loss function. By jointly optimizing these two functions, the W-Net architecture could extract segmentations in an unsupervised manner. After training, conditional random field smoothing was applied to improve segmentation quality. W-Net produced results comparable to the state-of-the-art unsupervised segmentation models, providing an alternative to the labor-intensive process required to produce datasets for supervised semantic segmentation.

In 2017, He et al. introduced Mask R-CNN, an alternative architecture that unified object detection, instance segmentation, and classification into a multi-task objective [44]. Expanding on the work of the Fast R-CNN [34] and Faster R-CNN [102], which explored only classification and object detection, the Mask R-CNN added a semantic segmentation path that was trained simultaneously with the classification and bounding box objectives. Unlike semantic segmentation, which labels all pixels of a class identically, this approach enabled instance segmentation, distinguishing between individual objects of the same class. This architecture achieved state-of-the-art performance on the common objects in context (COCO) dataset [73] and human pose estimation, showing the transferability and generality of the Mask R-CNN method.

Despite the success of Mask R-CNN and U-Net, the explosive success of transformer architectures in the language modeling domain (detailed in section **??**) inspired novel approaches to computer vision. Dosovitskiy et al. introduced the vision transformer (ViT) in 2020; instead of relying on convolutional layers, which were the standard for computer vision architectures, the authors leveraged the attention mechanism. By dividing images into 16 $\times$ 16 pixel "patches," and encoding these patches into tokens with a linear layer, the ViT architecture allowed images to be processed as sequences. This solution addressed many concerns in the original W-Net paper regarding data efficiency; while the original ViT was supervised, it established the architectural framework that later enabled powerful self-supervised masked imaged modeling (such as MAE), reducing reliance on expensive labeled data

[142]. Although not strictly a segmentation model, the ViT created a framework that enabled transformer-powered segmentation.

Building on the ideas introduced by the ViT, researchers at Meta released the Segment Anything Model (SAM) for semantic and instance segmentation [62]. SAM is a foundation model built with a ViT architecture, trained on over 11 million images and 1 billion segmentation masks with a model-in-the-loop paradigm. Newer versions, SAM 2 [101] and SAM 3 [**carion2025sam**] were released in 2024 and 2025, respectively, to enable video segmentation, improve inference speed, and enhance text prompting.

- unet: first fully convolutional semantic segmentation architecture; utilized skip connections, and reduction/reconstruction strategy to produce segmentation maps. allows precise localization by combining high-level semantic info with low-level texture info.
- wnet: unsupervised variant of U-Net; built on unet by taking using one u-net to produce a segmentation map and another to recreate the image, and then
- mask r-cnn
- ViT: first prominent vision architecture to borrow from language modeling, produces image-patch tokens and performs next-token predictions on those tokens for more effective unsupervised training. When paired with a decoder, semantic segmentation could be performed.
- SAM: scaled version of ViT by meta, foundational model trained on millions of images for general segmentation. more variants appeared for better segmentation, then video segmentation

1.2.4 Representation Learning and Transfer Learning in Vision

- SimCLR/ Contrastive learning
- CLIP: used contrastive learning to align language and image representations in vector space, matching image-caption pairs to each other with a contrastive loss function. core concept that applies itself to multimodal modeling, not only with images, but also with graphs, language, and other domains, as will be discussed.
- ViT: the unsupervised training process allowed for representation learning and fine-tuning without the burden of manually labeled datasets
- SAM: performed this representation learning on a massive scale, creating a foundation model that had learned general image representations for millions of examples

1.2.5 Transformer-Based Computer-Vision

- ViT
- SAM
- CLIP
- BLIP-2
- DINO
- MAE

1.3 Language Modeling

1.3.1 N-Gram

1.3.2 Neural Language Modeling

1.3.3 RNN/LSTM

1.3.4 Transformer

1.3.5 Scaling Laws

1.3.6 Multi-Modal Language Modeling

1.4 Graph Learning for Biology

1.4.1 Spectral vs. Spatial Methods

1.4.2 Message Passing Neural Networks (MPNNs)

1.4.3 Graph Attention Networks (GATs)

1.5 High-Performance Computing for Deep Learning

Explain why scaling is crucial for modern deep learning

1.5.1 Parameter Efficient Fine-Tuning (PEFT)

1.5.2 Distributed Data Parallel

1.5.3 Fully-Sharded Data Parallel

1.5.4 DeepSpeed ZeRO-3

1.5.5 Tensor Parallel

1.5.6 Pipeline Parallel

1.6 Deep Learning for Biology

While deep learning has achieved notable success in domains such as computer vision and natural language processing, its application to biological data remains comparatively less mature. Unlike image or text modalities, biological data exhibit complexity, hierarchical structure, and long-range dependencies, all with limited or noisy supervision. These characteristics challenge many of the assumptions that underlie standard deep learning architectures and training paradigms. As a result, biological problems provide compelling settings to extent how advances in representation learning, model scaling, and computational infrastructure generalize beyond areas where deep learning has been most successful. Understanding of prior work in vision, language modeling, and scalable training is therefore essential for situating and motivating the approaches explored in this thesis.

1.6.1 Vision-Based Phenotyping

1.6.2 Transformer-Based Genomic Prediction

1.6.3 LLM-Based Mechanistic Exploration

Chapter 2

A High-Throughput Deep Learning Pipeline for Image-Based Pennycress Phenotyping

2.1 Introduction

Pennycress (*Thlaspi arvense L.*) is a winter annual in the *Brassicaceae* family that has shown potential as a bioenergy feedstock and cover crop. Realizing this potential requires population-scale studies, which are currently blocked by phenotyping bottlenecks, most notably the manual collection of seed pod traits. Currently, no frameworks exist that enable scalable approaches to collecting detailed seed pod measurements. In this chapter, we introduce a deep learning pipeline for image-based phenotyping of pennycress, which we apply at population scale and extend through genome-wide association and mechanistic interpretation.

2.1.1 Pennycress and Plant Phenotyping

Pennycress has a high seed-oil content and non-food fatty-acid profile that support its use in biodiesel production without competing with the food supply. As a cover crop,

it grows in the corn-soy fallow window, provides agronomic services including erosion control and nitrogen capture, and integrates into existing rotations. Pennycress is also tractable for breeding due to its short generation time, established transformation protocols, and sequenced reference genome.

Improving these traits requires breeding pennycress for improved feedstock and cover-crop traits. Therefore, it is necessary to assess the standing genetic variation across natural accessions. *Plant phenotyping*, the process of measuring and characterizing traits such as seed yield, oil content, and flowering time, is the entry point for this effort, allowing variation to be connected to genotype, environment, or breeding targets. Measuring phenotypes on a population scale allows scientists to infer environmental and climatic influence on these traits and understand their genetic architecture. The most informative studies are both *extensive*, measuring traits across many accessions, and *intensive*, measuring many traits per sample.

In pennycress, however, intensive phenotyping at population scale remains out of reach. Pod-level traits such as wing area, envelope morphology, and seed count per pod are tedious to collect by hand [126] and are therefore systematically omitted from population-scale studies, despite their direct relevance to seed yield and pod-shattering resistance [84].

2.1.2 Deep Learning for Image-Based Phenotyping

Image-based phenotyping enables measurements at scales and resolutions that are infeasible by hand, from cell-level microscopy to satellite imagery [67]. A single imaging modality can yield many traits simultaneously, allowing researchers to extract complex topological and morphological features that are inaccessible with manual measurement. For pod-level phenotyping, the operative subtask is segmentation: distinguishing wing, envelope, and seed regions within each pod.

Classical segmentation approaches require controlled imaging conditions and hand-tuned thresholds, which do not generalize across natural variation in raw scans; deep learning models operate on raw images directly, learning relevant features end-to-end.

Deep learning models have achieved state-of-the-art performance in computer vision tasks ranging from image classification to segmentation [66, 41]. In the plant phenotyping domain, deep learning models have been applied to disease detection [**<empty citation>**], organ counting [**<empty citation>**], and seed phenotyping [**<empty citation>**]. Despite progress in adjacent phenotyping tasks, automated pod-level phenotyping in pennycress has not yet been addressed, leaving wing, envelope, and seed traits systematically unmeasured. In this chapter, we develop a deep learning pipeline for pennycress seed pod segmentation, enabling intensive and extensive phenotyping. We apply this pipeline at population scale and extend it through genome-wide association and mechanistic interpretation.

2.1.3 Study Objectives

This work pursues a three-stage research trajectory. First, we train and benchmark deep learning models to measure pennycress seed pods at population scale, selecting the best-performing architecture through systematic comparison. Next, we identify candidate genes through phenotype-driven genome-wide association. Finally, we derive and mechanistically interpret functionally coherent gene modules from those candidate genes. We completed an end-to-end pass through this pipeline. Our architecture benchmark (Objective **??**) confirmed that the U-Net baseline remained the best-performing model, so the existing phenotype set and downstream results stand without re-running. The remaining proposed work extends to the preliminary mechanistic interpretation beyond its current proof-of-concept stage. The five objectives below structure this trajectory:

1. **Establish a baseline deep learning segmentation pipeline for pennycress seed pods.** We trained a U-Net model with a boundary-weighted

loss function on a hand-annotated dataset of 345 seed pod images, providing a baseline against which we benchmarked modern architectures (Objective **??**).

2. **Characterize population-scale phenotypic structure of pennycress seed pods.** We applied the baseline model to over 11,000 seed pods across 771 pennycress accessions and extracted over 50 phenotypes per pod, assessing biological validity through trait distributions and known wing-seed associations. Building on these results, we constructed predictive phenotype networks (PPNs) over the extracted phenotypes using iRF-LOOP [18], identifying mutually predictive traits across the population.

3. **Identify the best-performing segmentation architecture for pennycress.** We benchmarked a panel of segmentation architectures against the U-Net baseline, including nnU-Net, DeepLabV3+, SegFormer, Mask2Former, and SAM 3. The U-Net baseline achieved the highest mIoU, so the existing phenotype set stands and the downstream analyses (Objectives **??** and **??**) were not re-run.

4. **Identify candidate genes from phenotype-driven genome-wide associations.** We ran a genome-wide association study (GWAS) to map phenotypes to genomic regions, and mapped significant SNP loci to their nearest annotated genes to construct a candidate gene set.

5. **Derive and mechanistically interpret gene modules from the candidate gene set.** We applied MENTOR [122] to the candidate seed gene set to derive functionally related gene modules, and we conducted a preliminary, proof-of-concept mechanistic interpretation of these modules using MENTOR-IA [130].

2.2 Background and Related Work

2.3 Methods

This section details the methods which underpin our phenotyping pipeline, spanning dataset construction, image segmentation, population-scale phenotype extraction, phenotype-to-genotype mapping, and module-level mechanistic interpretation. Section **??** covers dataset collection, annotation, and preprocessing. Section **??** presents the U-Net segmentation backbone and training procedure. Section **??** details the phenotype extraction stage. Section **??** presents the models and criterion for benchmarking. Sections **??**–**??** cover the downstream biological pipeline: PEN construction, phenotype-to-genotype mapping, module derivation with MENTOR, and mechanistic interpretation with MENTOR-IA.

2.3.1 Dataset

Data Collection

Our raw data consisted of 661 scanned images of pennycress (Thlaspi arvense L.) seed pods collected from a variety of plantations in the midwest, including those at Western Illinois University (WIU), Illinois State University (ISU), and University of Minnesota (UM). After collecting seed pods, we imaged them using an Epson V39 Perfection scanner. Each scanned image contains multiple seed pods due to their size. Figure **??** displays an example raw image scan. The 661 scans were drawn from a population of 771 pennycress accessions, with each scan containing approximately 15 seed pods, yielding a dataset of approximately 11,000 individual pods after separation (Section **??**). A subset of 345 of these seed pods were hand-annotated for supervised training and evaluation (Section **??**), while the full set was used for population-scale inference following model training.

Figure 2.1: Raw Pennycress Seed Pod Image

Data Annotation

For supervised model training, we created pixel-level annotated label data. We used GNU Image Manipulation Program (GIMP) to annotate sample data. We labeled three areas of each pennycress seed pod: the wing was annotated with red pixels, the envelope was annotated with green pixels, and the seeds were annotated with blue pixels. We annotated these areas due to the biological significance of their phenotypes. Figure **??** shows an example of a separated pennycress seed pod and its associated segmentation. We segmented 345 individual pods; we used 281 seed pods for training and validation and the remaining 64 for testing.

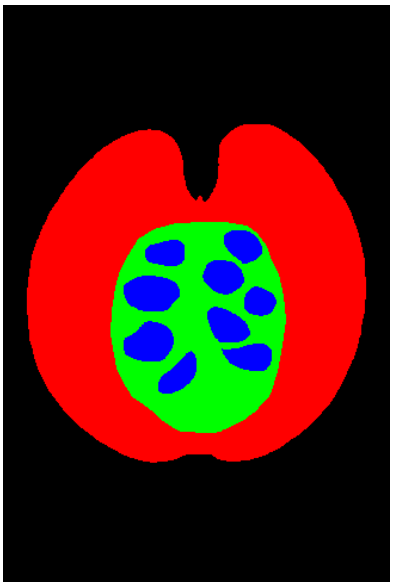
Data Preprocessing

We implemented two preprocessing steps required before model input. First, we separated each pennycress seed pod in the raw image. We created rectangular bounding boxes around each seed pod object in python, and we saved each bounding box as a separate image. We used these separated images for model input.

After separation, we padded each image with 128 pixels in all directions. Image padding enables us to generate tiles near the edge of images, allowing the model to see a greater context. We explain the tile generation process further in Section **??**. The red box in the leftmost image in Figure **??** highlights the padding boundary.

0.4
(a)
t

Figure 2.2: Separated Pennycress Seed Pod Image



0.4
(a)
t

Figure 2.3: Annotated Pennycress Seed Pod Image

Figure 2.4: Separated and Annotated *T. arvense* Images

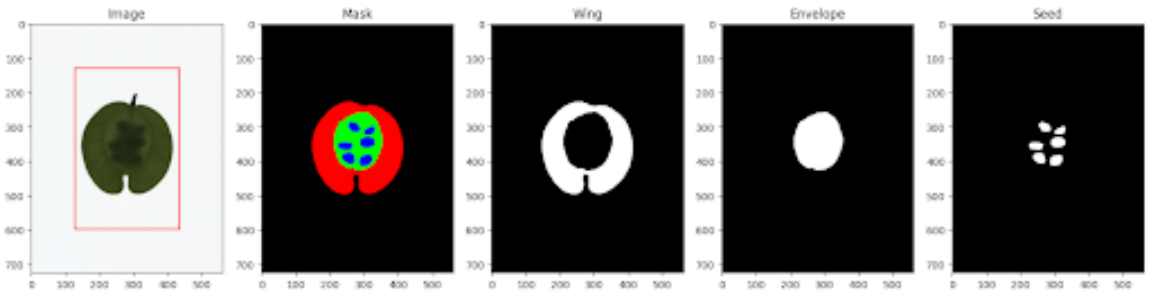


Figure 2.5: Preprocessed Pennycress Images

Image Tile Generation

We implemented a tile generation scheme to increase the amount of training data available and build an invariance to common imperfections that occur during the image collection process. First, we selected a window size of 128, meaning that the height and width of each tile was 128 $\times$ 128. Pixels on the seed pod were randomly initialized as a central pixel for our tiles. After the algorithm selected central pixels, it used the surrounding 128 $\times$ 128 pixels to create a tile.

During validation, we left tiles unchanged to prompt the most accurate predictions possible. However, during training, we applied various image augmentations and transformations to build an invariance to common data collection issues. We applied rotation, blur, saturation, brightness, flips, and transposes with a 50% chance of occurrence each, greatly increasing the number of unique types of tiles that could be generated.

We selected each type of transformation to combat certain issues that occur during the scanning process. For example, blur transformations simulate potential smudges on the scanner screen. Rotations model the differences in angle between scanned leaves, and brightness transformations capture the potential for shadows underneath folds in an image. By using these transformations to simulate real imaging issues, the model learns to become invariant to such features and instead focus on biologically relevant information. Therefore, the purpose of the tile generation scheme is twofold: not only does it increase the amount of available training data, but it also builds an

invariance to common scanning issues. We show examples of generated training tiles and their corresponding labels in Figure **??**.

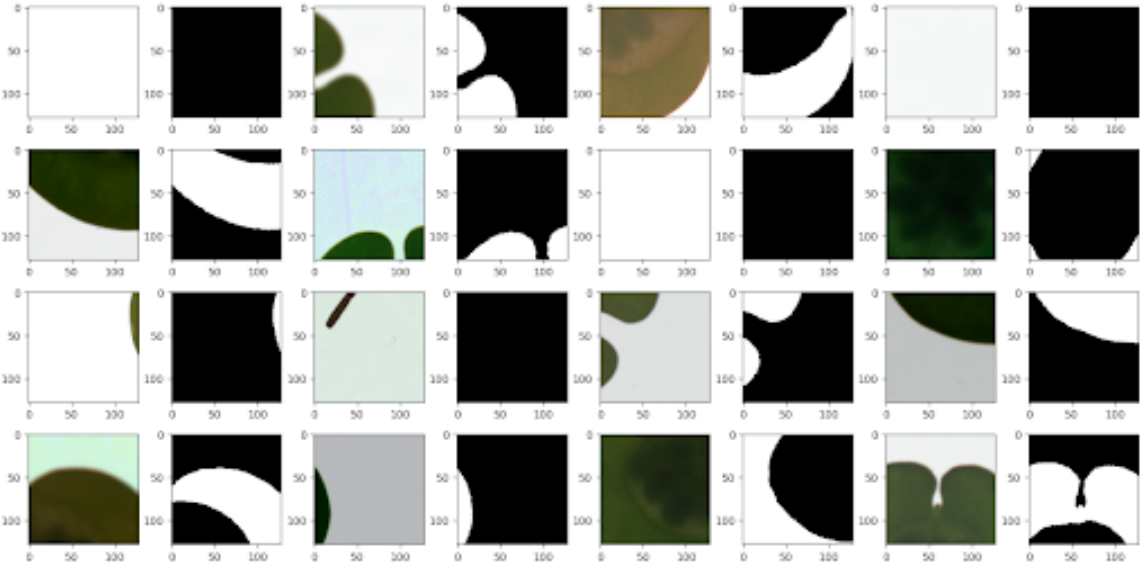


Figure 2.6: Generated Image Tiles

2.3.2 U-Net Baseline

Architecture

U-Net is a fully convolutional neural network architecture that established strong baseline performance for image segmentation performance across multiple domains. The model includes a contractive path to capture hierarchical and contextual features followed by a symmetric expansive path to provide specific localizations within the image [103]. We used a U-Net with similar architecture to [103] but modified the block structure, activation function, and various hyperparameters to improve domain-specific performance.

Since raw seed pod images were too large and computationally expensive to fit into model memory while training, we used the generated tiles created in Section **??** for training.

The model's input is a 3-channel RGB tile of size 128 × 128, and its output is also 128 × 128 with four channels corresponding to background, wing, envelope, and seed classes. We applied a softmax activation function to create a pixel-wise segmentation map, where the model gives each pixel a class label. We then compared segmentation maps to the corresponding hand-annotated segmentations detailed in Section **??** and minimized the weighted cross-entropy loss between them to train the model.

The main component in the U-Net architecture is a convolutional block. Each block contains one convolutional layer with a 3 × 3 kernel followed by a batch normalization, SeLU activation, and dropout layer with a rate of 0.1.

The U-Net contains seven convolutional blocks of varying sizes. The model has an identical symmetric architecture. The layer in the first block of the model contains a feature map of size 128 × 128; the layer has 32 kernels, extracting one feature for each kernel. The first four blocks in the U-Net model comprise the contractive steps; 2 × 2 max-pooling layers are used between blocks to halve the size of the feature maps, while the number of kernels in each layer is doubled. This process is repeated four times, ending the contractive step with a 16 × 16 feature map layer containing 128 kernels. The final block in the contractive step is not followed by a max-pooling operation; it is referred to as the bottleneck layer due to its low resolution and dense feature space.

The next three blocks in the network make up the expansive step. A 2 × 2 up-convolution layer doubles the feature map size before each block, while the number of kernels in each block decreases by a factor of two. These blocks are stacked until an output layer with a feature map of size 128 × 128 and 16 kernels each is produced. We apply a final convolutional layer with one kernel to the output to create a map of size 128 × 128 × 4, with each of the four channels representing a segmentation class. Additionally, we use skip connections to concatenate corresponding contractive and expansive layers. Not only do these connections prevent gradient vanishing, but they also leverage information from multiple levels of features in the image, allowing for

state-of-the-art segmentation performance. We visualize the model's architecture in Figure **??**.

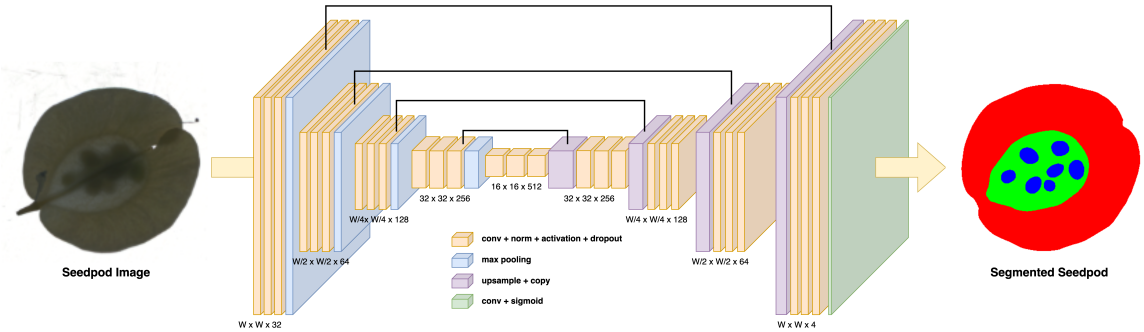


Figure 2.7: Pennycress U-Net architecture

Boundary-Weighted Cross Entropy Loss

To account for uncertainty in the exact boundaries of our ground-truth labels, we implemented a weighted cross-entropy loss that scales the per-pixel loss by a boundary-aware weight map.

We first compute a raw weight map from the eroded seed mask. We binarize the seed channel, subtract its binary erosion to isolate the boundary ring, and then calculate the Euclidean distance from each pixel to the closest border pixel to produce a distance map. We clip distances at a maximum of $b = 5$ pixels and normalize to the range $[0, 1]$. We evaluated two configurations parameterized by a border weight w_b.

In the **boundary down-weighted** configuration ($w_b < 1$), we min-max normalize the raw weight map so that pixel values $W_{i,j} \in [0, 1]$. We then invert the weight map so that pixels with a smaller distance to the boundary have a lower weight value. Finally, we scale the weight map so that its range is $W_{i,j} \in [w_b, 1]$. This configuration treats boundary annotations as the noisiest signal and lets the model rely more on confidently-labeled interior pixels.

In the **boundary up-weighted** configuration ($w_b > 1$), we instead clip the raw weight at a minimum of 1, yielding $W_{i,j} \in [1, w_b]$ so that distant pixels carry weight

1 and boundary pixels carry weight up to w_b. This configuration treats boundaries as the most informative regions to learn.

We then multiply the weight map pixelwise with cross-entropy loss and average over pixels:

$$L_{WCE}(t, p(\theta)) = -\frac{1}{\mid I \mid\mid J \mid} \sum_{j}^{J} \sum_{i}^{I} \sum_{k}^{K} W_{i,j}(t_{i,j,k} \log(p(\theta)_{i,j,k})), \tag{2.1}$$

where t is the ground truth segmentation map and $p(\theta)$ is the predicted segmentation map as a function of U-Net parameters.

Model Training

We trained the U-Net for up to 150,000 mini-batch updates using the Adam optimizer with a batch size of 32. Rather than partitioning training into fixed epochs over the tile dataset, we sampled tiles continuously from the train split (Section **??**) and tracked progress by iteration count. We evaluated validation loss every 1,000 batches and applied an early stopping criterion of 50 evaluation intervals: if the best validation loss did not improve over 50 consecutive checkpoints (50,000 batches), training terminated.

We used a learning rate schedule with linear warmup and cosine decay. The learning rate ramped from 0 to 1×10^{-3} over the first 1,000 batches to stabilize early gradients, then decayed via a cosine schedule to 1×10^{-5} over the subsequent 90,000 iterations and held constant thereafter.

2.3.3 Phenotype Extraction

We developed functions to extract 51 features from each seed pod segmentation map, grouped into three categories: absolute, ratio, or color.

Absolute Features

We calculated seven absolute features from each segmentation map: wing area and perimeter, envelope area and perimeter, seed area and perimeter, and seed count. Areas for each tissue were calculated summing the pixels assigned to that class. Perimeters were calculated using the Scikit-Image [132] on nested binary masks. Wing perimeter was calculated from the union of wing, envelope, and seed pixels, envelope perimeter was calculated from the union of envelope and seed pixels, and seed perimeter was calculated on the seed mask alone. Pixel counts were converted to physical area by dividing DPI^2 ($600^2 = 360,000$) and multiplying by 6.4516 cm^2/in^2.

In addition to measuring area and perimeter features, we utilized the watershed algorithm to count the number of seeds in each segmented image. First, we isolated the seed channel and applied a morphological opening to suppress noise. Then, we calculated the Euclidean distance from each seed pixel to the nearest non-seed pixel. We applied a threshold to this distance map, setting any pixel that was less than 60% of the maximum distance value in the map to 0. The thresholded foreground was labeled into connected components and passed as markers to the OpenCV [12] watershed algorithm, which returned a per-seed segmentation; seed count is defined as the number of unique markers in the map. The watershed process is visualized in Figure **??**.

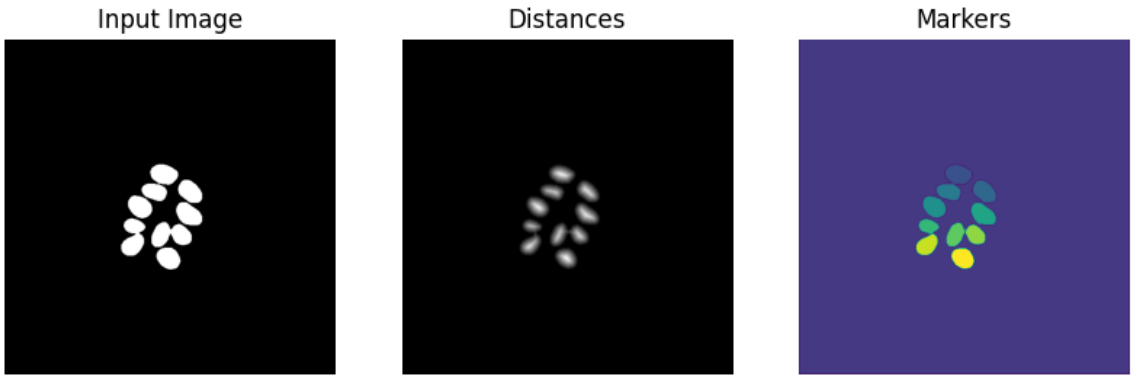


Figure 2.8: Watershed Segmentation of Pennycress Seeds

Ratio Features

We calculated 18 ratio features from the segmented seed pod masks, with six to-total ratios and twelve between-tissue ratios, evenly split between area and perimeter based variants. The six to-total ratios divide each tissue's area or perimeter by the total area or perimeter, yielding wing-to-total, envelope-to-total, and seed-to-total ratios for both measurement types.

The 12 between-tissue ratios cover all pairs of distinct tissues for both area and perimeter. These include ratios into seed (envelope/seed, wing/seed), into envelope (wing/envelope, seed/envelope), and into wing (seed/wing, envelope/wing), computed for both measurement types. Reciprocal ratios are retained (e.g., seed/envelope and envelope/seed) so that downstream analysis is not restricted.

Color Features

We extracted nine color features from each tissue: red, green, blue (RGB); hue, saturation, brightness (HSV); and lightness, a^* (green-red), and b^* (blue-yellow) from CIELAB. We converted each input image into all three color spaces using OpenCV [12]. We then created a tensor of these features with size $\texttt{height} \times \texttt{width} \times 9$. We used the segmentation masks to subset this tensor by tissue, yielding three sub-tensors covering wing, envelope, and seed features separately.

We aggregated each color channel across spatial dimensions by taking the mean over relevant tissue's pixels, producing a nine-dimensional color vector per tissue. Across the three tissues, this yielded 27 features in total.

2.3.4 Segmentation Architecture Benchmarking

To ensure that our pipeline extracts the most accurate downstream phenotypes possible, we sought to identify the most performant segmentation architecture. We tested six architectures across three families: CNNs, transformers, and foundation models.

We compared two CNN architectures to the baseline U-Net: nnU-Net [54] and DeepLabV3+ [14]. nnU-Net is an approach which configures a U-Net systematically through a three-stage pipeline. The selection process includes fixed parameters, which generalize across many datasets; rule-based parameters, which are determined by unique dataset and pipeline characteristics; and empirical parameters, which are learned through trial and error. Because nnU-Net derives its own patching, batch size, and augmentation scheme through fingerprinting and rule-based selection, the benchmarking for this architecture did not include our custom tile generation scheme (Section **??**). DeepLabV3+ combines an encoder-decoder architecture with atrous depthwise separable convolutions, balancing the use of long-range spatial context with fine-grained boundary detection.

We selected SegFormer [143] and Mask2Former [16] as exemplar transformer architectures for benchmarking. SegFormer uses a hierarchical transformer encoder backbone which generates both high-resolution, fine-grained features and low-resolution, coarse features, and a lightweight MLP decoder to fuse these features for the final semantic segmentation task. Unlike SegFormer, which optimizes a per-pixel classification objective, Mask2Former adopts an alternate paradigm: set prediction. Mask2Former uses a multi-scale encoder backbone to extract features, which are then upsampled to high-resolution per-pixel embeddings with a pixel decoder. Additionally, Mask2Former utilizes a transformer decoder with learnable query inputs to predict mask candidates. These queries are refined through cross-attention with the multi-scale features, and the transformer decoder output and pixel decoder output are combined via dot product to produce region predictions. Although Mask2Former is designed to be a general segmentation architecture, supporting panoptic, instance, and semantic segmentation, we trained only for a semantic segmentation objective due to our task formulation.

We evaluated SAM 3 [13] as a candidate foundation model for benchmarking. SAM 3 is a foundation model designed for promptable concept segmentation, taking input as either text or image exemplars. SAM 3 was trained with a model-in-the-loop

data engine on over 1 billion images, generating segmentation examples using human input while shifting towards more model reliance during the annotation process. Although SAM 3 is capable of segmenting video and using concept prompts, we focused on the benefits that its pretrained encoder provides, adopting an approach similar to ClassWise-SAM-Adapter [94] by using a lightweight convolutional mask decoder for classwise semantic segmentation. Furthermore, we applied Low-Rank Adaptation (LoRA) [49] to the image encoder to enable parameter-efficient fine-tuning and limit drift from pretrained representations.

We trained DeepLabV3+, SegFormer, and Mask2Former from default pretrained backbones with the same tile generation and preprocessing scheme that we used for our custom U-Net to ensure consistency. Since nnU-Net's data-aware preprocessing approach is emphasized as one of its strengths, we did not apply our custom preprocessing pipeline to it, instead using nnU-Net's adaptive fingerprinting setup for preprocessing and model training. As a consequence, nnU-Net scored predictions under its own pipeline at the level of individual seed pod crops (64 crops), whereas all other architectures were evaluated on the five full images that contain the same pods. nnU-Net's metrics were aggregated per pod rather than per image; the two are directionally comparable but not computed over identical units.

For all models except nnU-Net and Mask2Former, we trained variants with unweighted, up-weighted, and down-weighted boundary loss. Due to nnU-Net's adaptive pipeline and Mask2Former's mask-classification objective, boundary-weighting was not compatible with these architectures. These model families provide coverage across architectures (CNN and transformer), training objectives (per-pixel prediction and set prediction), and learning scenarios (supervised pretraining and foundation model pretraining).

2.3.5 Predictive Phenotype Networks

To explore the relationships between measured phenotypes, we produced PPNs using iRF-LOOP [18]. Given an input set of extracted phenotypes for each pod, we computed the average value for each accession, which we used as input to the model. iRF-LOOP uses iterative random forest to predict a single held out phenotype from all others. Iterative random forest modifies each feature's sampling probability according to its importance (e.g., mean decrease in Gini impurity) allowing informative features to be selected more frequently as the model converges. This process is repeated by holding out each phenotype, with the entire model output yielding pairwise importance values between phenotypes.

Using the iRF-LOOP output, we constructed predictive phenotype networks. PPNs take the form of a directed graph, where nodes represent phenotypes and edges represent predictive relationships. An edge from phenotype A to phenotype B indicates that A is predictive of B, with edge weight equal to the iRF-LOOP importance of A in predicting B. Using the raw iRF-LOOP output produces a dense network with all phenotypes connected to all others; however, we induced sparsity and highlighted only strong predictive relationships by retaining only edges whose weight falls in the top 5%. PPN outputs allowed us to perform an exploratory analysis of relationships between phenotypes, while also helping expand our seed set before downstream multiplex network analyses with MENTOR (Section **??**).

2.3.6 Genome-Wide Association Study

After measuring seed pod phenotypes and their relationships on a population scale, we perform a genome-wide association study (GWAS) to find significant relationships between phenotypes and pennycress loci.

Phenotype Curation and Heritability Estimation

Before running association tests, we assessed whether each phenotype had sufficient genetic signal to warrant GWAS by estimating broad-sense heritability (H^2) independently for each environment (dryland and floodland). Because most accessions in our panel are unreplicated, H^2 was estimated on the subset of $n = 25$ replicated genotypes. For each phenotype, we fit a linear mixed model with genotype as a random effect and computed

$$H^2 = \sigma_G^2/(\sigma_G^2 + \sigma_E^2),$$

where σ_G^2 and σ_E^2 are genotypic and residual variance components. Phenotypes with $H^2 < 0.05$ were removed from downstream association analyses due to lacking sufficient heritable signal. We additionally restricted our analysis to phenotypes derived from the U-Net segmentation pipeline described in Section **??**, excluding manually measured traits to maintain methodological consistency.

Association Testing

GWAS tests each variant across the genome for statistical association with a phenotype, typically using a linear mixed model that includes a kinship matrix and population structure covariates to control for confounding from relatedness and stratification. Each SNP-phenotype test produces a p-value, and SNPs exceeding a genome-wide significance threshold are reported as significant associations. We fit four single- and multi-locus association models implemented in GAPIT [133]: MLM, MLMM, FarmCPU, and BLINK. SNPs were filtered to a minor allele frequency (MAF) of ≥ 0.03 prior to testing. We ran GWAS independently per phenotype and a retained SNPs as significant if they passed a Benjamini-Hochberg false discovery rate threshold of $q \leq 0.2$ under at least one of the four methods. The permissive cutoff was justified by our use of GRIN [121] to filter false positives under the relaxed threshold.

SNP-to-Gene Assignment

After finding significant loci, we mapped SNPs to candidate genes using a two-pronged approach. For each significant SNP, we first fit a local linkage disequilibrium (LD) decay curve and defined a data-driven window around the SNP based on fitted decay distance. We then identified all genes whose coordinates fall within this window on the pennycress TAV2 reference genome. To capture associations with genes that lie outside the LD window but in close physical proximity, we additionally included the nearest annotated gene by physical distance. SNPs in regions where LD decay could not be reliably fit were excluded from LD-window assignment but retained via nearest-gene distance. Because functional annotation of the pennycress genome is limited, we annotated each assigned pennycress gene by its best BLAST-hit Arabidopsis ortholog from TAIR [32] to enable downstream biological interpretation.

Seed Gene Set Construction

We aggregate gene assignments across both environments by taking the union of genes mapped from significant SNPs, regardless of which environment or which GWAS method produced the original association. After applying the heritability filter ($H^2 > 0.05$) and restricting to phenotypes derived from the U-Net segmentation pipeline, four pod related phenotypes (`pod_env_b_2`, `pod_env_to_seed_area`, `pod_wing_area`, and `pod_wing_to_seed_area`) yielded significant GWAS hits, mapping to 6 unique SNPs and 69 pennycress genes, 64 of which had annotated Arabidopsis TAIR orthologs. We note that these four phenotypes are highly correlated, as they are geometric ratios computed from a common set of pod measurements, and therefore do not represent four biologically independent signals. This gene set serves as the seed input to the MENTOR module derivation described in Section **??**.

To expand the candidate gene set, we additionally included the direct neighbors of each surviving phenotype in the PPN (Section **??**) and took the union of their GWAS-derived gene assignments. This leverages the assumption that mutually predictive

phenotypes share some underlying genetic architecture. The resulting expanded seed gene set is passed to GRIN (Section **??**) for network-based refinement.

2.3.7 MENTOR Module Derivation

GRIN Filtering

Before using our GWAS-derived gene set as input to MENTOR for module construction, we applied GRIN [121] to filter potential false positive genes. Given a seed gene set of potentially significant genes and network of connected genes, GRIN operates in two stages. First, it runs RWR on the seed gene set, yielding a rank vector of the most closely related genes in the network to the seed genes; in parallel, it runs RWR on 100 randomly sampled gene sets of the same size as the seed set, creating a null distribution rank vector. Next, GRIN uses a sliding-window Mann-Whitney U test to evaluate the likelihood of observing the seed gene RWR ranks at random. With this sliding window, a threshold can be defined where the ranks become insignificant, providing a threshold for filtering. The resulting significant genes are kept as members of the seed gene set, while all others are dropped. We applied GRIN filtering to our seed gene set to eliminate potential false positive significant genes.

Module Construction

Using our filtered set of seed genes as input, we applied MENTOR to group them into functionally related modules. MENTOR is a multi-step framework for deriving functionally related modules given a multiplex network and seed genes as input [122]. We used an *Arabidopsis thaliana* multiplex, as the species is closely related to pennycress and has a fully sequenced genome.

Starting from the GRIN-filtered seed genes (Section **??**), MENTOR applies random walk with restart (RWR) as a diffusion method to measure the probability of traveling to each gene in the multiplex from all genes in the seed set. Each seed gene produces its own score vector over all multiplex nodes, with indices representing

the probability of reaching each node from the seed gene. The score vectors are then rank-ordered, truncated at the elbow of the mean score vs. rank curve, and converted into a dissimilarity matrix by taking $(1 - \rho)$, with ρ representing Spearman correlation, between each pair of genes' rank-ordered score vectors. With the resulting dissimilarities, MENTOR creates a dendrogram using agglomerative clustering with complete linkage as our module distance measure. This dendrogram is then thresholded to examine modules of a certain size. To make exploration tractable, we chose to only focus on modules that contain genes from our seed set.

2.3.8 MENTOR-IA Interpretation

After deriving a set of modules with MENTOR, we conducted a preliminary, proof-of-concept interpretation of these modules using MENTOR-IA [130]. MENTOR-IA is a multi-agent LLM framework for mechanistic interpretation, in which specialized agents each contribute to a distinct line of evidence. These agents include a mygene interpreter, which utilizes gene summaries, GO terms, and pathway annotations to explore each gene's role individually; a literature agent, which explores mechanistic connections in related literature via Europe PMC; an enrichment agent, which tests whether a module is enriched for pathways in GO, KEGG, or Reactome; a network agent, which uses subgraph evidence to refine the interpretation; and a meta aggregator, which synthesizes evidence to produce a coherent mechanistic summary.

Using the evidence provided by MENTOR-IA as a starting point, we manually examined pennycress and *Brassicacae* literature to assess whether the proposed mechanisms were consistent with known biology. For the modules we examined, the proposed mechanisms aligned with established pod- and seed-trait biology, demonstrating that the framework can connect the phenotypes measured by our pipeline to candidate mechanistic explanations.

2.4 Results

2.4.1 Evaluation Metrics

We used intersection over union (IoU), also known as the Jaccard index, as the primary evaluation metric for this project. Given a ground truth pixel-wise segmentation set A and a predicted pixel-wise segmentation set B for a single class on a single image, the IoU is defined as:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{2.2}$$

IoU lies in $[0, 1]$, with a value of 1 indicating identical sets and 0 indicating disjoint sets. We report class-wise IoU for each tissue class (wing, envelope, seed), reflecting the differing biological importance and segmentation difficulty of each class. We additionally report the mean IoU (mIoU) as a single-number summary used to rank models during benchmarking. For consistency, we compute IoU identically across all architectures.

2.4.2 Segmentation Benchmark

We evaluated our models' segmentation performance on 5 raw images containing 61 seed pods in total. Per-class IoU and across-class mIoU are reported in Table **??**. The boundary down-weighted U-Net achieved the highest mIoU at 85.58%, narrowly ahead of the baseline U-Net at 85.53%. nnU-Net underpeformed compared to the custom U-Net variants on envelope and seed classes. We attribute this gap to nnU-Net's incompatibility with our custom tile generation scheme: nnU-Net imposes its own patch sampling policy, and the resulting losses diverged early in training, indicating overfitting due to a comparatively small effective sample size. No modern architecture surpassed the U-Net baseline. The transformer- and foundation-model-based architectures matched U-Net on the large wing class but trailed on the more

Model	Wing	Envelope	Seed	mIoU
U-Net	95.60%	82.84%	78.14%	85.53%
U-Net-Down	95.53%	**82.94%**	**78.28%**	**85.58%**
U-Net-Up	95.58%	82.24%	77.60%	85.14%
nnU-Net	95.09%	79.92%	72.48%	82.50%
DeepLabV3+	95.19%	81.53%	75.80%	84.17%
DeepLabV3+-Up	95.28%	81.70%	75.64%	84.21%
DeepLabV3+-Down	95.14%	81.31%	75.89%	84.11%
SegFormer	95.03%	80.45%	75.07%	83.52%
SegFormer-Up	94.99%	80.22%	75.23%	83.48%
SegFormer-Down	95.31%	81.68%	76.36%	84.45%
Mask2Former	**95.65%**	82.80%	76.65%	85.03%
SAM 3	95.15%	76.42%	65.62%	79.07%
SAM 3-Up	95.35%	76.78%	68.15%	80.09%
SAM 3-Down	94.95%	73.49%	57.61%	75.35%

Table 2.1: Segmentation benchmark results. Per-class IoU (wing, envelope, seed) and mean IoU (mIoU); mIoU is the mean of the three tissue-class IoUs. Bold values denote the best score in each column.

detailed envelope and seed classes, with SAM 3 weakest overall (seed IoU as low as 57.61%). The pattern may reflect our fixed small-tile pipeline, which limits the large receptive fields and pretraining priors these models rely on (Section **??**). We found no prior work measuring semantic segmentation of pennycress to compare with our model benchmark suite.

2.4.3 Population-Scale Phenotyping

Having established a baseline segmentation model (Objective **??**), we applied it at population scale to characterize phenotypic structure across the pennycress accession panel. We measured phenotypes from over 11,000 seed pods across 771 accessions. Each image contained approximately 15 seed pods and took around 2 minutes of model inference time to segment, compared to approximately 1.5 hours of hand segmentation per image, yielding an estimated $30\times - 45\times$ throughput gain depending on annotator pace.

We used the feature extraction pipeline described in Section **??** to construct an open-source dataset of over 50 phenotypes for all 11,000+ seed pods. Figure **??** shows the distributions of three key phenotypes: wing area, envelope area, and seed area. These traits correspond directly to the three annotated tissue classes, making their distributions a useful biological plausibility check on segmentation output. Each trait was log-normally distributed with distinct means and variances, consistent with the log-normal distributions commonly observed in quantitative plant traits [59].

Consistent with the wing-seed regulatory association described in Section **??**, wing area was positively correlated with seed area and seed count (Pearson $r = 0.51$ and 0.30, respectively; Figure **??**). The resulting per-accession phenotype matrix served two roles: as input to iRF-LOOP for PPN construction (Section **??**), and as the candidate trait pool for heritability filtering and GWAS (Section **??**).

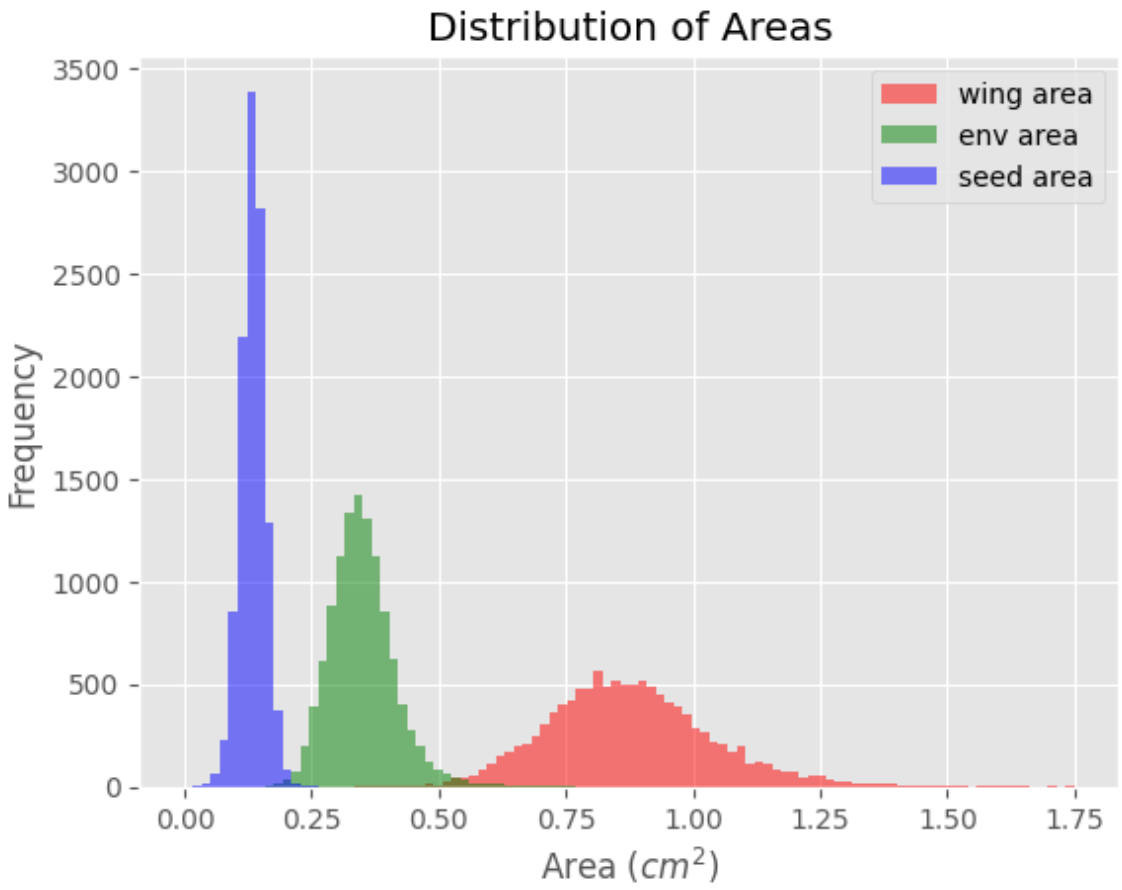


Figure 2.9: Distribution of wing, envelope, and seed areas across the over 11,000 seed pod population. All three traits exhibit log-normal distributions consistent with expected plant trait variation.

2.4.4 Predictive Phenotype Network Construction

After extracting phenotypes from our dataset, we used iRF-LOOP to construct PPNs for use in our downstream mechanistic interpretation. Running iRF-LOOP on the entire dataset yields a PPN with 48 nodes and 108 directed edges after filtering the network to include only the top 5% of relationships (Figure **??**). Despite being a directed network, the PPN exhibits high reciprocity, indicating that predictive relationships tend to be mutual, which is expected for many geometrically derived traits.

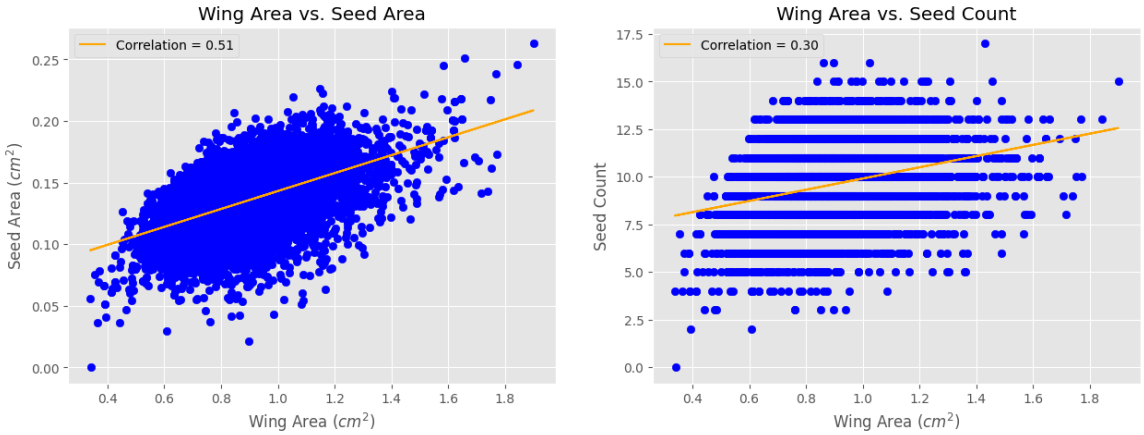


Figure 2.10: Wing versus (a) seed area and (b) seed count across the population. Positive correlations (Spearman $r = 0.51$ and 0.30, respectively) are consistent with wing-seed regulatory association in Brassicaceae.

The PPN separates into three connected components after filtering, each dominated by a single feature type: a 27-node component containing all color features across all three tissues, a 12-node component containing absolute geometry and area ratios, and a 9-node component containing all perimeter ratios. This separation by measurement type is expected given how tightly color and geometric features covary within each class.

The network's structure offers a biological plausibility check on the extracted phenotypes. We use the PPN to expand the candidate gene set prior to module derivation: for each phenotype carrying a significant GWAS association, we add the genes mapped from its predictively related (PPN-neighboring) phenotypes, broadening the seed set beyond the directly associated traits.

2.4.5 Phenotype Curation and Heritability estimation

During mechanistic interpretation, we limit our focus to heritable phenotypes derived by our U-Net pipeline in order to isolate the pipeline's utility. Starting with a full panel of 149 phenotypes collected from the same areas described in Section **??**, we drop all traits that were not extracted via our pipeline. Next, we calculate heritabilities

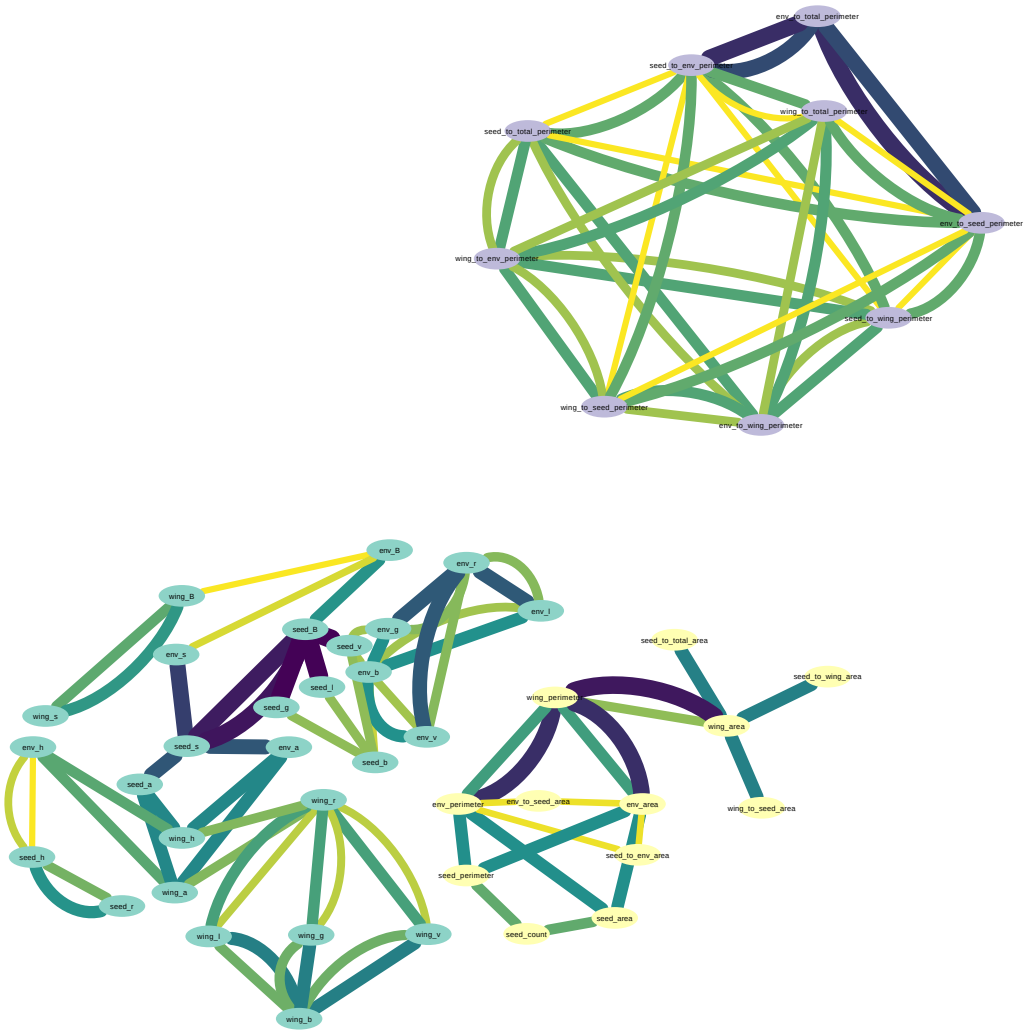


Figure 2.11: Predictive Phenotype Network for Pod-Related Traits. The PPN contains 48 nodes and 108 directed edges after the network is filtered to include only the top 5% of predictive relationships.

using a mixed model and remove all phenotypes with $H^2 > 0.05$. This yields 38 heritable traits for genome-wide association testing.

2.4.6 Genome-Wide Association Testing

We performed GWAS on the heritability-filtered, U-Net derive pod traits on a panel of 188 accessions with dryland and floodland environments tested separately. We

used a four GAPIT models to estimate significance. Variant with MAF below 0.03 were excluded, and we used an FDR (BH) threshold of ≤ 0.20 as our cutoff.

GWAS yielded 10 model-level association calls representing six unique SNPs across four phenotypes: *pod_env_b_2*, *pod_env_to_seed_area*, *pod_wing_area*, and *pod_wing_to_seed_area*. All of the retained associations came from the dryland environment, and the most recurrent signal came from *pod_env_to_seed_area*. Our GWAS results are summarized in Table **??**.

Trait	H^2	SNPs	Calls	Models	Lead SNP	Lead raw p
`pod_env_b_2`	0.217	1	1	MLMM	`4_8127380`	8.71×10^{-7}
`pod_env_to_seed_area`	0.200	3	7	BLINK, FarmCPU, MLMM	`5_963441`	1.04×10^{-7}
`pod_wing_area`	0.366	1	1	MLMM	`7_63065228`	4.14×10^{-7}
`pod_wing_to_seed_area`	0.285	1	1	MLMM	`2_2343403`	6.06×10^{-7}

Table 2.2: Summary of retained GWAS associations for heritable U-Net-derived pod traits. Model-level calls are counted as unique SNP-trait-environment-method combinations. All retained associations were detected in the dryland environment.

2.4.7 SNP-to-Gene Mapping

We mapped the six retained SNPs from GWAS to pennycress genes using the mixed LD-decay and distance-based approach described in Section **??**. For each SNP, we defined a window centered on the SNP with a width set by local LD-decay distance and assigned the nearest annotated genes falling in that window, retaining any gene whose body directly contained that SNP. Windows spanned approximately $24.6 - 49.6\tilde{\text{k}}\text{b}$, with strong LD-decay fits ($R^2 = 0.944$–0.989), reflecting the different recombination contexts of each chromosomal region.

This procedure produced 69 candidate pennycress genes, with half of the loci tracing to `pod_env_to_seed_area`, and each of the remaining three loci tracing to the remaining three traits. We mapped these genes to their nearest Arabidopsis orthologs in TAIR [32] to enable cross-species functional annotation and module derivation.

This mapping yielded 64 unique orthologs, as several pennycress genes shared an a single Arabidopsis ortholog. This ortholog set constitues the candidate seed gene set carried forward to module derivation (Section **??**).

2.4.8 MENTOR Module Derivation

We applied MENTOR to the GRIN-filtered candidate seed gene set to resolve it into functionally related modules using the process described in Section **??**. GRIN filtering reduced the set of 64 candidate orthologs to 33 seed genes, pruning 31 genes that lacked sufficient network support. Cutting the dendrogram at a starting module count of three under a maximum module size of 40 resolved our 33 seed genes into three modules of 5, 11, and 17 genes (Figure **??**). No module exceeded the size cap, so no sub-clustering was triggered. We then used these modules for mechanistic interpretation (Section **??**).

2.4.9 MENTOR-IA Preliminary Interpretation

We used MENTOR-IA to conduct a preliminary interpretation of the three modules derived by MENTOR (Section **??**). Module 1, which contained 18 Arabidopsis orthologs, was run with the enrichment agent, literature agent, mygene interpreter, and meta aggregator. Module 2, which contained 11 orthologs, was run with the same agents as Module 1. Module 3, which only contained 5 orthologs, was run with only the mygene interpreter and meta aggregator, as the small gene count limited the power of enrichment analyses. Due to the lack of an associated network for our modules, we did not use the network agent for analysis.

MENTOR-IA produced per-agent summaries for each module, along with a final interpretation that synthesized evidence across agents. This interpretation included keyword themes, enrichment statistics from GO and KEGG, and a mechanistic summary with candidate supporting citations. Across modules, the proposed mechanisms recovered known *Brassicaceae* pod and seed biology rather than arbitrary

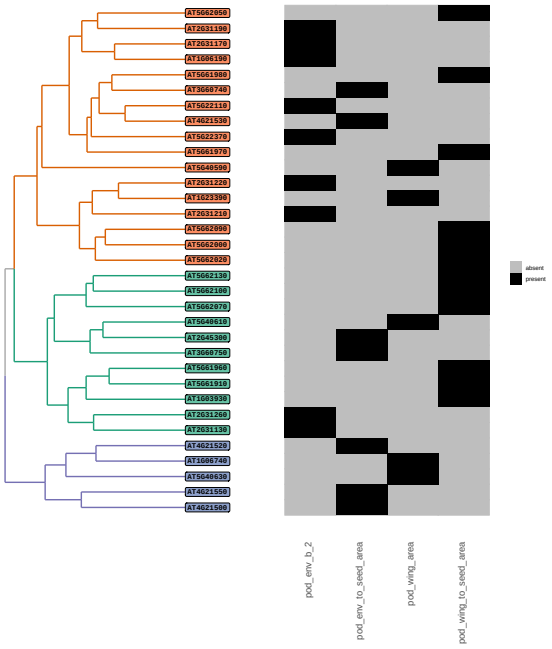


Figure 2.12: Rectangular Layout

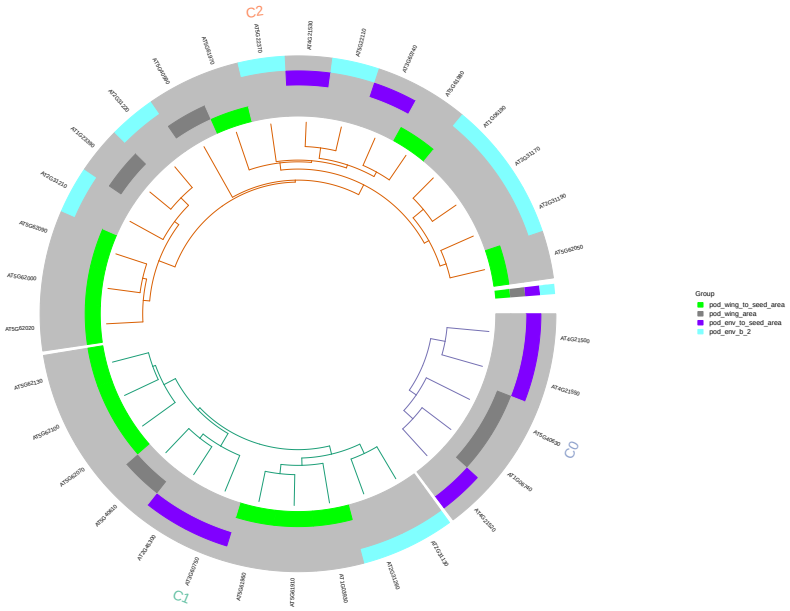


Figure 2.13: Polar Layout

Figure 2.14: MENTOR dendrogram of the 33 GRIN-refined orthologs, shown in (??) rectangular and (??) polar layout. Hierarchical clustering of RWR embeddings resolves the gene set into three modules (5, 11, and 17 genes), indicated by leaf color.

associations, demonstrating our pipeline's ability to connect measured phenotypes to candidate mechanistic explanations. We manually cross-checked the agent output against pennycress and *Brassicaceae* literature, finding the mechanistic summaries to be consistent with reported biology.

Module 1 Interpretation

- topic sentence - module theme (auxin-centered control of organ size, reproductive development)
- enrichment evidence to verify
- "hero mechanism": ARF2 represses auxin responsive genes + limits cell proliferation in the integuments; loss of ARF2 enlarges seeds and other organs. acts with SEEDSTICK and GORDITA during integument/seed development. coherent candidate mechanism for variation in $\text{pod}_e nv_t o_s eed_a reaandpod_w ing_a rea[schruff2006, paol$
- RUS2 sustains polar auxin transport, consistent with auxin flux shaping, planar wing growth. [Ge 2010]
- optional: ARF2 regulates senescence, which can modulate chlorophyll loss and pigmentation
- trait linkage + hedge: module recovers an auxin/seed-size axis that aligns with GWAS-identified pod traits –¿ consistent with established Arabidopsis biology as a candidate mechanism, but not pennycress specific

Module 2 Interpretation

- TS - metabolic supply –¿ chloroplast carbon, energy metabolism, autophagy-mediated nutrient remobilization. plausible connection to pod filling, envelope-to-seed allocation

- enrichment: limited to a single term, response to calcium ion. remaining genes derived from gene-level annotation instead of enrichment
- strongest thread (autophagy): ATG9 sustains autophagic flux, which recycles nitrogen and carbon to developing seeds [inoue 2006, shin 2014, kang 2018, luo 2023, lai 2020]. candidate influence on pod$_e nv_t o_s eed_a rea$
- TKL1 links the calvin cycle to the shikimate/pentose-phosphate routes [Rocha 2014; Khozaei 2015], and GPDHc1 balances NADH/NAD+ [Shen 2006], consistent with carbon partitioning shaping biomass and wing area
- calcium-response enrichment via IQD23 suggests Ca2+ signaling may coordinate cytoskeletal remodeling during pod expansion
- metabolic supply hypothesis: trait connection is more indirect than module 1's candidate, and is once again arabidopsis derived

Module 3 Interpretation

Since module 3 yielded only sparse gene-level annotation and lacked enrichment and literature support, we did not advance a mechanistic interpretation for it; its genes remain candidates for future analysis.

2.5 Discussion

Our team developed a U-Net model for high-throughput segmentation of pennycress seed pods, and we created an open-source, hand annotated ground truth dataset of 365 seed pod images to use for training data. We performed a population-scale study with the model, extracting over 50 phenotypes from 11,000 seed pods belonging to 761 naturally occurring accessions of pennycress. The U-Net produced over 95% accurate wing and envelope segmentations and almost 80% accurate seed

segmentations. Additionally, model inference provided a 30x speed increase over manual hand-segmentation.

Despite this progress, our work has some limitations. Collecting and scanning seed pods to use for segmentation is still a labor-intensive process; this could become a bottleneck in future population-scale studies. Additionally, the model's performance is dependent on the quality of label data due to its supervised nature. Accurately annotating seed border pixels was difficult due to the low brightness and contrast levels in image scans, and this limitation was reflected in the lower accuracy for seeds than other surfaces. Therefore, it is important to find methods to improve image quality in future work, possibly utilizing higher-resolution scanners.

This work has several implications for future pennycress research. We found no prior work that extracts intensive phenotypes from pennycress seed pods due to the labor-intensive nature of this task. Therefore, this article provides a novel pipeline for extracting several biologically important phenotypes at the population-scale for pennycress. The framework allows scientists to perform experiments on pennycress that were previously impossible; for example, researchers can examine how treatments and experimental conditions affect entire populations by measuring seed pod phenotypes across treatment groups. Furthermore, scientists can test hypotheses about phenotypic relationships on a broad scale, leading to higher-impact research. Additionally, the pipeline can be used in conjunction with GWAS to pinpoint the underlying genetic drivers of traits. Scientists can find relationships between genome regions and phenotypes collected from the pipeline, providing a path for new genetic modification studies in the future. Overall, the pipeline presented in this paper provides several avenues for the advancement of pennycress as a seed-oil based biofuel feedstock.

Chapter 3

G × E Transformer

3.1 Introduction

Genotype-by-environment (G × E) interactions are a key mechanism underlying crop stability and performance [[CITE]]. In turn, accurately modeling these interactions has important practical implications, including increased food security, development of alternate fuel sources, and optimized farming practices [[CITE HERE]]. However, due to to the amount of heterogeneous data required to model interactions, G × E prediction is a task that requires a diversity of knowledge, including genetics, agronomy, statistics, and computer science.

Early genomic prediction approaches relied on small, single-environment datasets and statistical models that primarily captured additive effects, such as genomic best linear unbiased prediction (GBLUP) and Bayesian linear mixed models. While effective for narrow settings, these methods struggle to scale to heterogeneous data and capture non-linear interactions among loci and environments.

An increase in publicly available data has brought both renewed focus and challenge to GxE prediction. The Genomes to Fields (G2F) Maize G × E prediction challenge formalized this focus by curating a public dataset of genotypes and

environments, containing over 150,000 observations across 10 years and hundreds of field sites [136].

Deep learning models are well-suited for genomic prediction because they can capture dependencies on the local and global level across a heterogeneity of data sources. Due to the expansion of publicly available data along with innovations in consumer graphics processing units, it is becoming increasingly more feasible to apply deep learning to G × E prediction [add comment about previous deep learning projects]

In this work, we developed a deep learning model, trained on the g2f maize prediction dataset, to predict maize grain yield. We used a multi-input structure to model the interaction between linkage disequilibrium, global genotype, and environmental covariates.

3.2 Background and Related Work

3.2.1 G2F Maize Competition

- G2F Paper [136]: Introduces the G × E prediction competition, where contestants predicted the absolute grain yield for maize hybrids on an open source dataset. The competition garnered a variety of contestants from various backgrounds and countries, and a variety of methods were used in the competition.

- Leveraging data from the Genomes-to-Fields Initiative to investigate genotype-by-environment interactions in maize in North America: Curates, cleans, and expands data from the G2F dataset.

- Multi-trait/environment sparse genomic prediction using the SFSI R-package: Authors introduce a framework for sparse genomic prediction across multiple

traits and environments. The model unifies sparse selection indices and sparse genomic prediction to solve sparse prediction problems.

- Using machine learning to integrate genetic and environmental data to model genotype-by-environment interactions:
- Nucleotide Transformer [21]
- SNVFormer [25]
- TF-Based Prediction with a knowledge module [141]
- DNABERT [57]
- DNABERT-2 [147]
- Caduceus [111]

3.3 Methods

3.3.1 Data

Data for this model were obtained through the Genomes2Fields (G2F) Maize Prediction Competition [136]. The dataset included trait, metadata, soil, weather, genotype, and environmental covariate files spanning 21 field sites across 10 years. We trained our model on data from 2014 to 2022, validated model performance on 2023, and tested on the competition's 2024 test set. Our train split contained 150,000 samples, while our validation and test splits contained around 9,500 samples each.

Although we had a large number of samples, data quality was variable across years and field sites. As the authors state in the competition's documentation:

"While we have curated and lightly filtered the data for quality we have **PURPOSELY** left much of the data in raw formats. There is also extensive missing data. These are the realities of working with large agricultural datasets!" [136]

Extensive preprocessing, including filtering and imputation, was necessary to address inconsistencies and ensure reliable performance. We detail the steps taken to clean our marker and environmental data in sections **??** and **??** respectively.

Marker Data

Genotype data were provided as single nucleotide variant (SNV) markers. Details about data collection are specified in the competition documentation [136].

Each diploid marker sequence consisted of 2240 loci. At each locus, alleles were encoded according to the scheme in Table **??**. A value of 0 represents two major alleles, a value of 1 represents a heterozygous pair of alleles (major-minor), and a value of 2 represents two minor alleles.

Allele Combination	**Value**
Major-Major (AA)	0
Major-Minor (Aa)	1
Minor-Minor (aa)	2

Table 3.1: Genomic Input Marker Encodings

Marker data preprocessing consisted of filtering and imputation. We excluded genotypes lacking marker data and removed loci with more than 10% missing values. Remaining missing loci were imputed using a parent-based median strategy: genotypes were grouped with others sharing at least one parent, and missing loci were replaced with the group median.

Environmental Data

The competition provided four categories of environmental data: trait data, soil data, weather data, and environmental covariates. Additionally, the competition organizers provided metadata with descriptions of each variable. More details on environmental data collection can be found in the competition manuscript [136].

Trait data, representing yield outcomes, served as the prediction target. We excluded all rows which lacked either yield measurements or complete environmental data.

We pruned highly correlated environmental features (those with a Pearson correlation coefficient > 0.98) using a correlation network approach. Each feature was represented as a node, and the edges connected features that exceeded the correlation threshold. The feature with the highest degree in each correlated cluster was retained. In the case of equal degree, one feature was selected at random. This approach allowed us to retain the signal provided from correlated variables while reducing our feature space.

After dropping missing and correlated features, we performed imputation on the weather station data included in the competition dataset. For weather stations with partially missing data, we replaced missing observations with the mean latitude/longitude data of all existing years. One research station, location TXH4, had no location data present. We used the Texas Tech New Deal Research farm [1] as a proxy for this weather station to retrieve the coordinates for this station, since both are located in Lubbock, Texas. We used the facility's official website to retrieve the New Deal Research Farm's latitude and longitude.

Following filtering and imputation, we integrated our location and environmental covariate datasets into a unified set of 374 variables. Continuous variables were standardized to zero mean and unit variance, and the scaler was fit only on the training set to prevent data leakage.

3.3.2 Model Architecture

We developed a multi-input model to predict maize yield as a function of both genotype and environment. The architecture comprises three specialized encoder blocks: a transformer to model long-range dependencies among genetic markers, a convolutional neural network (CNN) to capture local linkage disequilibrium, and a

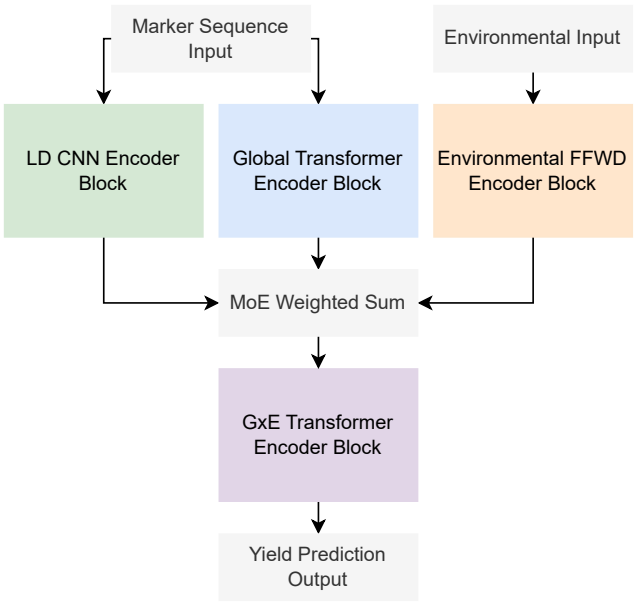


Figure 3.1: Overall Model Architecture.

feed-forward network (FFN) to represent environmental covariates. The outputs of these encoders are combined through a learned weighted-sum mechanism and passed to a genotype–by-environment (G×E) transformer module, which models interactions between genetic and environmental features. An overview of the full architecture is shown in Figure **??**.

Marker-Encoding Transformer

We used a transformer block to capture global dependencies in our input marker sequences (Figure **??**). Long-range dependencies across loci are critical in genomic prediction, as interactions among distant variants can influence phenotypes. We follow the architecture defined by GPT-2 [97] for our transformer blocks with slight architectural changes: we apply dropout after both our multi-head attention and feed-forward layers to reduce overfitting, and we apply layer normalization before our input to these layers instead of after.

We constructed our model input to handle the genotypic input that encodes allele frequency. Each of the 2240 loci was one-hot encoded into three channels corresponding to the allele states specific in Table **??**, yielding an input tensor shape of $(2240, 3)$. We embed the sequence input with both a trainable marker embedding layer and a static sine-cosine positional embedding layer, as introduced in [127].

Since the goal of our model is to encode a sequence instead of predict future tokens, we utilized bidirectional attention [24], allowing the transformer block to capture related tokens both before and after a token in the sequence.

The transformer encoder produced an output tensor of size (2240, *embedding dimension*). The embedding dimension is a user-specified parameter that is fixed across all model components. In our best performing model, we used an embedding dimension of 256.

Linkage Disequilibrium Encoding CNN

We used a CNN to encode linkage disequilibrium (LD) in our genotype input. CNNs are well-suited for capturing local autocorrelation [69], making them a natural choice to model LD along marker sequences. We implemented an architecture inspired by ResNet's residual blocks [42], but used a one-dimensional convolutional kernel due to the shape of our input.

As with the transformer block, our input shape is (2240, 3), where each channel represents the one-hot encoding vector for a locus. The first layer in our LD encoder is a convolutional "projection" layer, used to reshape the channel dimension from size three to our desired embedding dimension via the number of kernels. We keep the embedding dimension equal across all blocks in our model.

After projecting our input sequence to the desired embedding space, we utilize a residual block structure inspired by [42]. Our block structure consists of a one-dimensional convolution layer with *embedding dimension* kernels and "same" padding to retain the original input shape, followed by group normalization, ReLU activation and dropout. A post-activation residual connection also connects the input and

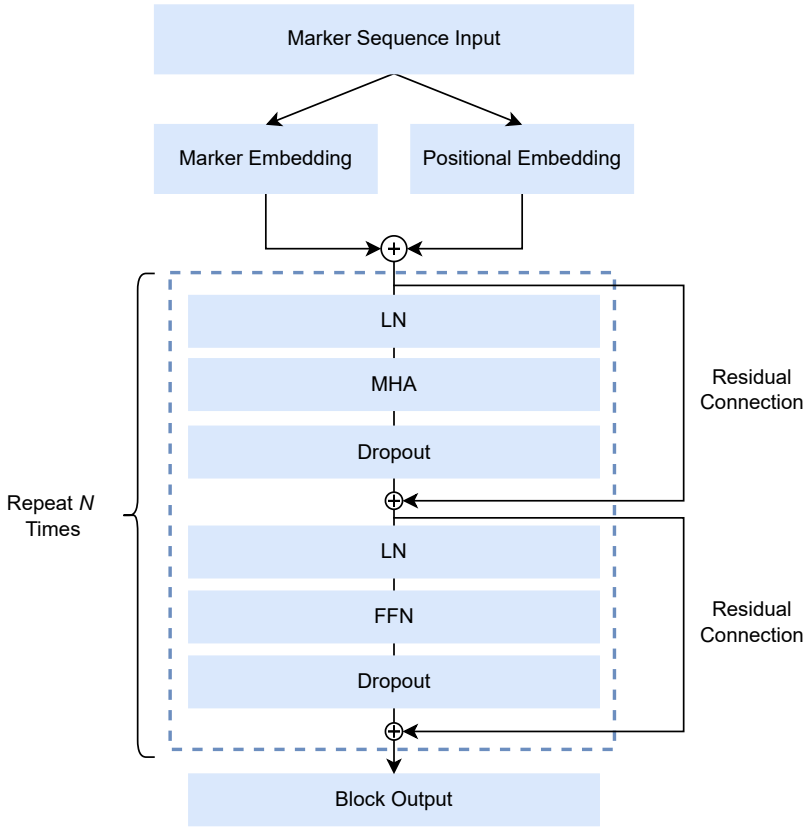


Figure 3.2: Transformer for Encoding Long-Range Genotype Dependencies

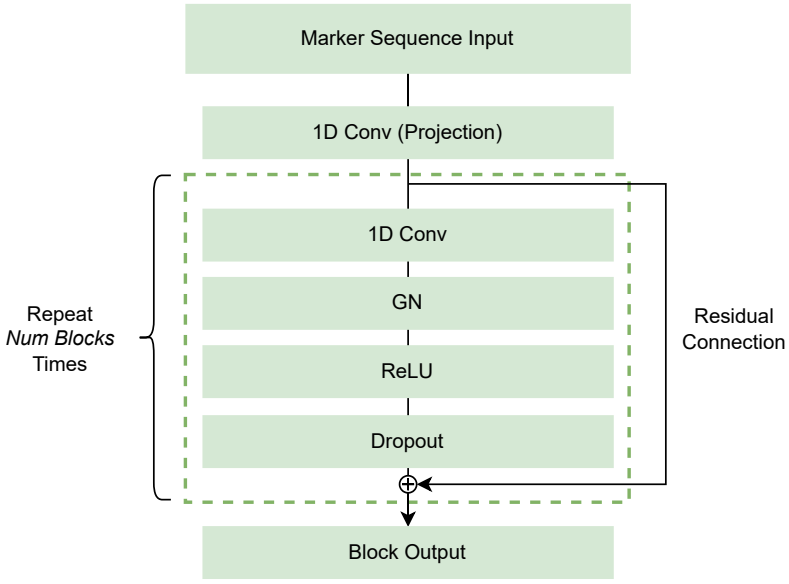


Figure 3.3: Convolutional Neural Network for Encoding Linkage Disequilibrium in Marker Sequences

output of each block. We treat the number of LD-encoding blocks in our model as a hyperparameter. Our best model had $N = 2$ blocks; deeper CNNs did not improve performance in our preliminary experiments.

The CNN output has size (2240, *embedding dimension*); the output retains the same number of marker indices as the input, but each marker representation is higher-dimensional. Since we keep the embedding dimension equal across all blocks, our best model's LD embedding dimension was also 256.

Environment Encoding Feed Forward Neural Network

We used a feed-forward neural network (FFN) to process our 374 environmental covariates. The network has a stacked block structure with residual connections, visualized in Figure **??**. We chose to encode environmental covariates with an FFN since there was no spatial or long-distance relationship between variables. Inputs were standardized to zero mean and unti variance before entering the FFN.

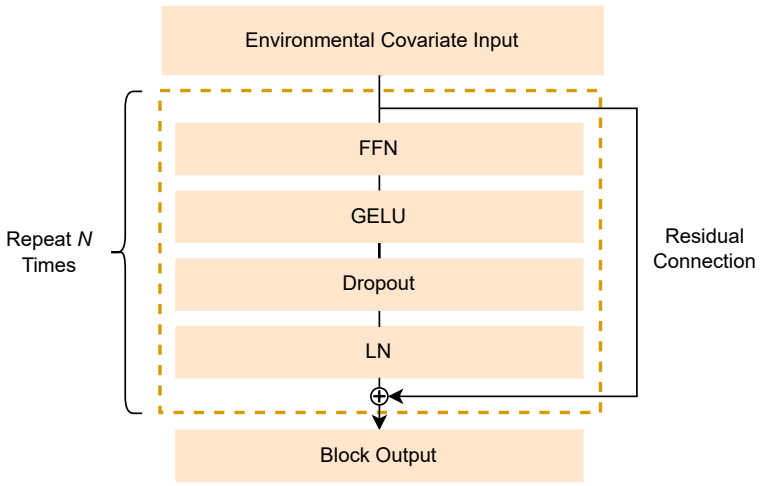


Figure 3.4: Feed-Forward Network for Encoding Environmental Covariates

Our FFN block consisted of four layers: a linear layer, GELU activation, dropout, and layer normalization. As with the transformer block, we included a post-activation residual connection to improve gradient flow and allow for deeper architectures [42]. The number of stacked FFN blocks was treated as a hyperparameter; in our best-performing configuration, we used $N = 1$.

Our linear layer projected the output to the same embedding dimension used across all encoders. Our block output was of shape (*embedding dimension*). Because environmental covariates lack a sequence dimension, the encoder output was broadcast along the sequence axis (length 2240) to align with the marker representations, yielding a final block output of (2240, *embedding dimension*).

G × E Encoding Transformer

We utilized a second transformer block to calculate a G × E vector representation for each sample given our CNN, Transformer, and FFN inputs. The architecture is visualized in Figure **??**; the transformer block structure exactly matches the genomic sequence transformer detailed in Section **??**.

To combine input representations, we used a learned weighted sum mechanism. Each encoder output was scaled by a single learned scalar weight which was broadcast across the sequence and embedding dimensions. This weighting scheme allows the model to scale the importance of each encoder prior to calculating the G $\times$ E interaction. Additionally, using only a single parameter for weighting each block increases the model's interpretability, as it provides a coarse estimate of importance for each individual encoder based on the magnitude of these weights. Our weighted sum layer produces an output of size (2240, *embedding dimension*).

The weighted sum output is then propagated through our G $\times$ E transformer block, which calculates the long-range dependencies between our encoded tokens. Although our layer structure for this transformer block is the same as the architecture specified in Section **??**, we double the amount of transformer blocks to $N = 2$ in our best-performing model. This increases the amount of parameters dedicated to G $\times$ E calculation, which is important to capture the highly non-linear relationship between these factors. Our transformer block produces an output of the same shape as its input: (2240, *embedding dimension*).

To obtain a scalar yield prediction, we reduced the G $\times$ E output to a fixed-length vector representation and mapped it to a single neuron. To perform this reduction, we take the mean across our sequence dimension, which has the effect of capturing the "average representation" across all loci. This reduces our output to a vector of size (*embedding dimension*), which we then propagate through a fully-connected layer with an output size of one to produce our yield prediction.

3.3.3 Training

Multitask Optimization

We trained our model to jointly optimize two tasks: Pearson correlation (**??**) and mean squared error (**??**). Because PCC values range from -1 to 1, subtracting PCC from 1 reframes the maximization of correlation as minimization of loss, with

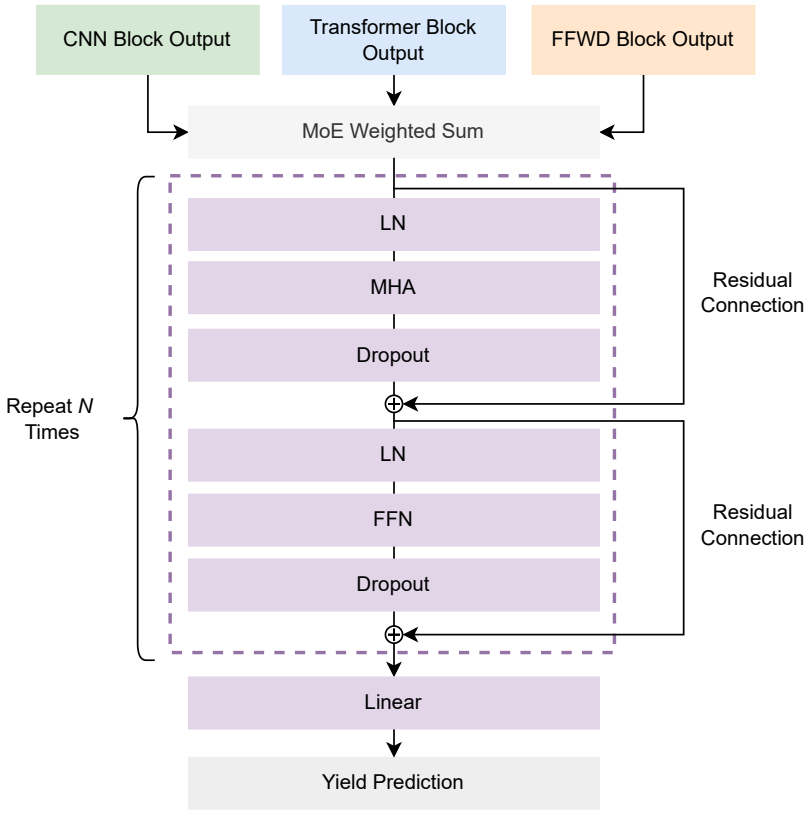


Figure 3.5: Transformer Block for Encoding G × E Interactions.

higher correlations producing lower loss values. The PCC loss conditions our model to capture the variation between samples in various environment-years, while MSE allows the model to correctly scale the data.

$$L_{\text{PCC}} = 1 - \left(\frac{\sum (y_i - \bar{y})(\hat{y}_i - \hat{\bar{y}})}{\sqrt{\sum (y_i - \bar{y})^2} \sqrt{(\sum \hat{y}_i - \hat{\bar{y}})^2}} \right) \tag{3.1}$$

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{3.2}$$

Additionally, due to the varying magnitudes of our objective functions, we introduced a scaling parameter, α, to our joint loss function. In our best training run using joint optimization, we set $\alpha = 1 \times 10^{-5}$. Our joint optimization objective is seen in Equation **??**.

$$L_{\text{Total}} = \alpha L_{\text{MSE}} + (1 - \alpha) L_{\text{PCC}} \tag{3.3}$$

Training Details

We utilized Oak Ridge National Laboratory's Frontier Supercomputer to train our model. To expedite training and implement larger global batch sizes, we implemented distributed data parallel (DDP) training. We trained on eight compute nodes with four AMD MI250X gpu accelerators (eight GPU dies per node exposed as devices) each. For our best model, we set the minibatch size to 256, yielding a global batch size of $8 * 8 * 256 = 16,384$. Our best model run was trained for a maximum of 5,000 total epochs with an early stopping callback of 200 epochs monitoring validation loss to prevent overtraining. In reality, our best training runs ended between 500 and 1,500 epochs. In addition to early stopping, we implemented a learning rate schedule similar to that specified in [97], where we set a learning rate warmup time of 10 epochs, and a cosine decay schedule from $1e^{-4}$ to $1e^{-6}$, with the minimum reached at the midpoint of training. We used the AdamW optimizer [75] with a weight decay parameter of $1e^{-5}$ to add regularization to our weights.

3.3.4 Evaluation

The competition's evaluation metric is the average PCC between predicted and observed yield values across all locations in the 2024 test set. Each environment contributed equally, regardless of sample size, and missing and unclean observations were dropped by the competition organizers prior to releasing the ground truth values. Since the evaluation code was closed source, we recreated the evaluation scheme based on the competition documentation [136]. We calculated the PCC for each environment in the 2024 dataset using `scipy.stats.pearsonr` from the `scipy` package [129], and we averaged the PCC values across environments to produce a total PCC value. The pseudocode for our algorithm is defined in Algorithm **??**.

each environment e in test set $y \leftarrow$ true yields for e $\hat{y} \leftarrow$ predicted yields for e $PCC_e \leftarrow \text{Pearson}(y, \hat{y})$ $PCC_{total} \leftarrow \frac{1}{E}\sum_{e=1}^{E} PCC_e$

3.4 Results

G Layers	E Layers	LD Layers	G×E Layers	Weighted Sum	Loss Size	Test PCC	Dropout	Heads	Embed
1	1	2	2	Yes	PCC	0.15	4	256	**0.496**
1	1	2	2	No	Both ($1e^{-5}$)	0.25	4	256	0.481

Table 3.2: Hyperparameter combinations and model results

3.5 Discussion

Team Name	Competition PCC
ORNL Comp. Bio	**0.49556**
Naaa	0.45005
NJUST_KMG1	0.44402
Model 2022 [136]	0.43766
wer2	0.43766
PARaBra	0.43710
transform(Base)	0.42605

Table 3.3: 2024 G × E prediction competition results

Chapter 4

MENTOR-RL

4.1 Introduction and Motivation

4.1.1 Biological Interpretation as Mechanistic Discovery

Biological interpretation involves the development and discovery of mechanistic relationships between biological entities such as genes, proteins, RNA, and other molecular components. Interpretation is essential for discovering context-specific relationships and developing models of disease, phenotypes, pathways, and regulatory systems. Recently, machine learning (ML) and artificial intelligence (AI) techniques such as iterative random forest (iRF) [8, 18], **Mentor** [122], and related approaches have accelerated research through the construction of biologically informed networks, which scientists explore to generate mechanistic and functional hypotheses.

As these methods increase the resolution and scope of networks, biological interpretation has become increasingly mediated by large, densely connected, multi-scale graph structures. To comprehensively examine these large biological networks and supporting databases, mechanistic exploration requires iterative querying and multi-step reasoning. Recent agentic interpretation systems such as eubiota [76] and Mentor-IA [130] demonstrate that provenance-preserving retrieval and tool-mediated synthesis can yield high-fidelity mechanistic narratives. However, these

pipelines depend on frontier model inference and are not fully optimized as a trainable policy.

4.1.2 Biological Interpretation is Bottlenecked by Human Cognition and Frontier Model Cost

Although ML/AI methods have refined mechanistic focus into groups of functionally related clades of genes, these clades can still be massive and intractable for individual researchers to systematically explore. Additionally, most research emphasizes a deep-but-narrow focus on a small subset of functionally related genes, which can overlook broader, yet biologically meaningful, network context, particularly in highly interactive systems such as living organisms. As biological networks grow in complexity, the combinatorial space of possible multi-hop interactions expands rapidly, while human interpretive capacity remains fixed.

While large language models (LLMs) can synthesize large volumes of information, frontier inference cost scales with exploration length and reasoning depth, limiting agentic deployment for long-horizon network traversal. This mismatch creates a scaling limit in biological mechanistic interpretation: rich relational structures can be computed, but downstream synthesis remains constrained by human bandwidth and model cost. Consequently, the speed, breadth, and contextual depth of mechanistic discovery are limited, particularly for complex traits, polygenic diseases, and multi-layer regulatory systems.

4.1.3 AI-Guided Reasoning to Extend Mechanistic Interpretation Beyond Human Scale

Open-source LLMs present an opportunity to aid scientists in mechanistic discovery tasks without being cost prohibitive. Recent developments in agent-based systems enable models to invoke deterministic tools with structured interfaces, substantially

reducing hallucination and enabling provenance-preserving exploration. We propose **Mentor-RL**, a reasoning agent trained on human-annotated biological ground truth and verifiable interaction objectives, enabling scalable mechanistic exploration without reliance on proprietary models.

By training a transformer-based agent with reinforcement learning across tool-use tasks and contextual levels of biological network data, we aim to produce a model that can traverse complex networks, synthesize *multi-hop* mechanistic hypotheses, and integrate both local functional structure and global topological context. Leveraging open-source model offerings allows local deployment, with training costs amortized across many exploration sessions. Rather than replacing scientific judgment, **Mentor-RL** extends researcher interpretive bandwidth, enabling broader, faster, and more context-aware mechanistic discovery.

We make four contributions. First, we formalize mechanistic graph exploration as a sequential decision process with a single policy operating in two modes: an actor that proposes reasoning and tool calls, and a verifier that updates a structured hypothesis and emits a continuation decision. Second, we construct a CORUM-grounded training corpus that supports four task cases: partial group completion, mechanism explanation, noisy group refinement, and recognition of no shared mechanism. Third, we design a deterministic, schema-grounded reward system over structured state transitions, enabling reproducible scoring without proprietary judges at the step level during preference optimization and the trajectory level for policy gradient optimization. Fourth, we propose a staged training pipeline, including warm-start SFT, DPO on shared-prefix trajectory branches, and GRPO for long-horizon exploration. We pair this with a curriculum that scales task difficulty, noise, and trajectory length. We will release the training environment, CORUM split indices, evaluation harness, and trained checkpoints, and we will benchmark against frontier and open models under both empirical and blind expert-preference evaluation.

4.2 Background and Related Work

Transition paragraph

4.2.1 Biological Networks and Mechanistic Ground Truth

Systems Biology and Mechanistic Discovery

Systems biology examines biological function by studying interactions between molecular entities such as genes, proteins, and metabolites, rather than analyzing their individual properties [40, 7]. A central goal is identifying *modules*, or groups of functionally related entities, and to characterize the mechanisms that underlie their behavior. Because network interactions can span across multiple scales and regulatory layers, modern systems biology relies on multiplex and multilayer network representations to capture context-specific relationships [6, 52]. Module-level analysis has become a foundational tool for identifying disease mechansisms, prioritzing drug targets, and interpreting polygenic phenotypes [80], motivating the need for curated, mechanistic ground truth data against which computational methods can be developed and evaluated.

Curated Interaction Maps

Large-scale systems biology depends on curated molecular interaction databases that capture experimentally supported relationships at varying levels of resolution. Creating high-coverage, expert-annotated networks is crucial for enabling large-scale systems biology studies and encouraging mechanistic discovery. BioGRID [90] catalogs physical and genetic interactions across humans and model organisms from both low- and high-throughput studies. Reactome [81, 33] provides manually curated, reaction-level pathway models for human biology. While both resources supply rich interaction evidence, neither organizes proteins into functional groups with verified co-membership. The CORUM database [35, 125] fills this gap by curating experimentally

verified mammalian protein complexes exclusively from low-throughput experiments. Additionally, entities are annotated with Gene Ontology terms at both the subunit and complex level. By providing verified group membership, mechanistic annotation, and expert-verified provenance, CORUM is uniquely suited as a ground-truth resource for training and evaluating mechanistic discovery systems.

Network Construction and Exploration on Multiplexes

Despite their value as annotated, benchmarkable datasets, curated interaction maps do not cover the full interactome; as a consequence, developing new network construction and exploration methods is crucial. Iterative random forest (iRF) expands on traditional random forests by weighting the feature sampling distribution according to variable importance scores across successive fits and using a stability selection procedure to capture higher-order interactions [8]. iRF has been used to construct predictive expression networks (PENs), which model gene-gene regulatory relationships inferred from co-expression, uncovering true connections from massive datasets that may be missed in hand-curated databases [18, 131]. Because biological systems comprise many complementary interaction modalities, integrating them without losing information motivated the development of the multiplex network formalism [83, 22], which represents the same node set across multiple edge layers. These structures remain compatible with classical exploration techniques; multiplex networks still support network propagation, such as random walk with restart (RWR) [63, 19] and other community detection methods built on the modularity objective [86], most prominently the Louvain method [10] to discover functional modules. More recently, the power of graph deep learning has also been realized, supplanting hand-engineered propagation techniques with learned ones in some data-rich regimes. Graph convolutional networks [61] aggregate neighbor features through message passing, while graph attention networks learn neighbor weights through self attention [128]. These techniques enable end-to-end supervised learning on tasks such as node classification, link prediction, and disease gene prioritization.

4.2.2 Tool-Using Language Models for Scientific Reasoning

Transformer Reasoning with Tool Use

In addition to deep learning's growing utility for biological discovery, neural language models have shown promise for complex reasoning, especially when grounded with tool use. [137] demonstrated that prompting a model to show its reasoning steps before answering improved multi-step task performance in a few-shot setting, and [64] extended this result to show its effectiveness in a zero-shot scenario. Furthermore, adding self-consistency mechanisms [134] and branched reasoning paths with lookahead and backtracking [145] have enhanced model reasoning capabilities.

Despite this progress in model reasoning, probabilistic decoding and training data cutoffs still leave models prone to errors, including hallucination. In turn, equipping models with the ability to use external tools during training or inference provides a layer of deterministic verification and enables interaction with the outside world. Toolformer [110] showed that self-supervised fine-tuning of tool calls yielded better performance with lower hallucination across several benchmarks, and Gorilla [93] extended this paradigm to handling large API vocabularies with retrieval. ReAct [144] showed that combining reasoning techniques with tool-based actions improved complex reasoning performance, and PAL [31] converted reasoning into programs, showing improved performance on computational tasks. These works demonstrate the power of combined reasoning and tool use for complex, multi-step problem solving, but their applications to scientific reasoning remain underexplored.

Agentic Architectures with Verification

While reasoning and tool use techniques improve task solving capability, they lack inherent verification mechanism, which is crucial for making factual scientific claims. Several works have explored self-verification through various methods. Self-Refine [77] and Reflexion [115] both define an iterative reflection loop without weight updates to improve answer quality; while self-refine uses single model to generate, critique, and

refine its own output, Reflexion utilizes sequential refinement, storing reflections in a persistent verbal memory. Self-RAG [3] fine-tunes a language model to emit reflection tokens that govern when to retrieve passages and how to assess their relevance and support. Eubiota [76] extends the self-verification process to multi-agent systems, using a dedicated verifier with separate weights to check the groundedness and factuality of other agent outputs. These approaches establish self-verification as a core component of agentic systems, but none combine a shared-weight actor-verifier paradigm with reinforcement learning over deterministic, structured state rewards, which we develop in Section **??**.

Agentic Gene Set Interpretation

LLM-based gene set interpretation has progressed from precomputed embeddings to fully agentic systems. GenePT [15] employs GPT-4 to create gene-level and cell-level embeddings from natural language for use in downstream interpretation tasks. GeneAgent [135] utilizes a multi-step GPT-4 pipeline with self-verification and retrieval from biological databases to provide a human-readable interpretations for experimentally derived gene lists. **Mentor-IA** [130] extends this paradigm by creating separate agents for mechanistic interpretation subtasks, such as literature search, enrichment analysis, network analysis, and verification, unifying these outputs into a single mechanistic summary. Eubiota [76] functions similarly to **Mentor-IA**, but with a focus on the human gut microbiome. The authors partially train a planning agent with GRPO while keeping proprietary model-powered subagents frozen. While these works share **Mentor-RL**'s agentic framing for mechanistic interpretation, none implements a fully trainable, end-to-end policy over verifiable rewards, which we introduce in Section **??**.

4.2.3 Reinforcement Learning for Long-Horizon Tool-Use

Preference Optimization

Christiano et al. introduced deep reinforcement learning from human feedback (RLHF) [17], replacing hand-specified reward functions with a value model trained on pairwise human preferences. This framework was extended by InstructGPT [91], which applied RLHF to align large language model outputs with human preferences. Direct preference optimization [98] built on this work by deriving a mathematically equivalent objective that operates directly on preference pairs, only using the policy and a reference while foregoing the reward model. Azar et al. unified RLHF and DPO under ΨPO, a general preference optimization framework, and introduced IPO to address DPO's tendency to overfit when observed preferences are deterministic or near-deterministic [5]. Hong et al. proposed ORPO, a single-stage alternative that merges preference alignment and SFT via an odds-ratio penalty, removing the need for a separate alignment stage [47]. While these methods carry varying tradeoffs, we elect to train **Mentor-RL** with DPO because our deterministic runtime supports scalable preference pair generation and the resulting checkpoint serves as a preference-aligned reference policy in the GRPO stage.

Policy Gradient and Group-Relative Methods

While preference optimization aligns LLMs to pairwise judgements, policy gradient methods optimize directly against scalar rewards over rollouts, supporting online learning across a wide range of tasks. Proximal policy optimization (PPO) [112] was introduced as a simple, efficient, and scalable alternative to TRPO [113], using a clipped probability ratio term to form a pessimistic surrogate of performance, preventing excessively large policy updates. In RLHF [91], this is paired with a token-wise KL penalty against a reference policy to anchor the model to its supervised initialization. Group relative policy optimization (GRPO) [114] builds on PPO, computing the advantage term by measuring the deviation of outputs from a group

mean rather than using a trained value model. Removing critic network roughly halves the trainable parameters of relative to PPO, making large-scale training more feasible. DeepSeek-R1 [39] demonstrated that GRPO with verifiable rewards can elicit emergent long-horizon reasoning behaviors at frontier scale. Having no learned critic, group-normalized advantages, and compatibility with deterministic verifiable rewards makes GRPO conducive to **Mentor-RL**, where the reward is a structured, schema-grounded signal over multi-hop biological graph exploration.

RL for Tool-Augmented Agents

In addition to its usefulness for general LLM training, RL and adjacent post-training methods has been extensively used to train LLM agents in tool use scenarios. WebGPT [85] used a fine-tuning, PPO, and rejection sampling pipeline to train a language agent for browser use and citation-backed question answering. ReST^{EM} [119] frames self-training from synthetic data as expectation maximization, demonstrating that fine-tuning on model-generated, filtered data can outperform fine-tuning on human-labeled datasets in verifiable reward settings. Toolformer [110] demonstrated that LLMs can use external tools by sampling candidate API calls during pretraining and retaining those whose outputs reduce the loss on subsequent tokens. Building on this self-supervised paradigm, ToolLLM [96] trained a language model for multi-step, multi-tool question answering on over 16,000 APIs with imitation learning. Agent-Q [95] utilizes monte carlo tree search and process supervision to construct model trajectories, which are used to optimize an agent via DPO. These works provide methodological precedent for **Mentor-RL**; however, we utilize on-policy trajectory generation and deterministic rewards grounded in expert-curated targets as a source of truth.

4.3 Proposed Methodology

Our methodology has five components. Sections **??** and **??** formalize the iterative exploration loop and the mechanistic interpretation task. Sections **??** and **??** describe dataset construction and environment design. Sections **??**, **??**, and **??** define the training pipeline, reward functions, and curriculum. Finally, Section **??** describes evaluation and benchmarking.

4.3.1 Agent Loop Formulation

Loop Formalization

We model mechanistic graph exploration as a sequential decision process executed by a single policy model with an internal self-verification step. The loop begins when the user provides a query, q, and optional evidence e^{user}, which may include text context, seed genes, or a multiplex network. These inputs initialize the working interpretation I_0 and the structured state S_0.

For each decision point $t < T_{\max}$, the current decision context is

$$D_t = (q, e^{\text{user}}, I_t, S_t).$$

Given D_t, the agent emits an action consisting of a textual reasoning step r_t and an optional tool action a_t. The reasoning step specifies the current subgoal or interpretation update, while the tool action specifies a runtime-executable call. If a tool action is emitted, it is executed by a deterministic runtime ε; otherwise, the observation is empty.

After the action is produced and any deterministic tool calls are executed, the same policy enters a verification mode. Conditioned on the query, user-provided evidence, the current interpretation, the current structured state, and the current reasoning/action outputs, the verifier updates the interpretation and structured state,

yielding (I_{t+1}, S_{t+1}). It also emits a continuation decision,

$$z_t \in \{\texttt{continue}, \texttt{revise}, \texttt{stop}\},$$

which determines whether the current interpretation should be extended, revised, or terminated. The decision `revise` indicates that the next iteration proceeds by correcting the interpretation and structured state rather than simply extending them.

The process repeats until either $z_t = \texttt{stop}$ or the maximum decision budget $T_{\max}$ is reached. At termination, I_T and S_T are returned as outputs. I_T provides a human-readable answer to the query with respect to the accumulated evidence, while S_T stores the corresponding verifiable outputs for reward computation and provenance preservation.

We formalize one iteration of the agent loop as

$$r_t, a_t \sim \pi_\theta^{\text{act}}(\cdot \mid D_t), \qquad o_t = \varepsilon(a_t), \qquad I_{t+1}, S_{t+1}, z_t \sim \pi_\theta^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where r_t is the agent's textual reasoning or subgoal, a_t is the optional tool action, and o_t is the observation returned by the deterministic runtime ε, with $\varepsilon(\varnothing) = \varnothing$ by convention. The policies π_θ^{act} and $\pi_\theta^{\text{verify}}$ denote two prompting modes of the same underlying model, sharing weights so that biological vocabulary and tool-use competence learned in one mode transfers to the other. We treat the risk that the verifier collapses as a monitoring concern during training, discussed in Section **??**. At each decision point, the pair (I_t, S_t) represents the current mechanistic hypothesis, with I_t serving as a human-readable view and S_t as a machine-readable view of the same underlying state. The full agent loop algorithm is detailed in Appendix **??**.

Working Interpretation

The working interpretation I_t is the human-readable form of the current mechanistic hypothesis and contains:

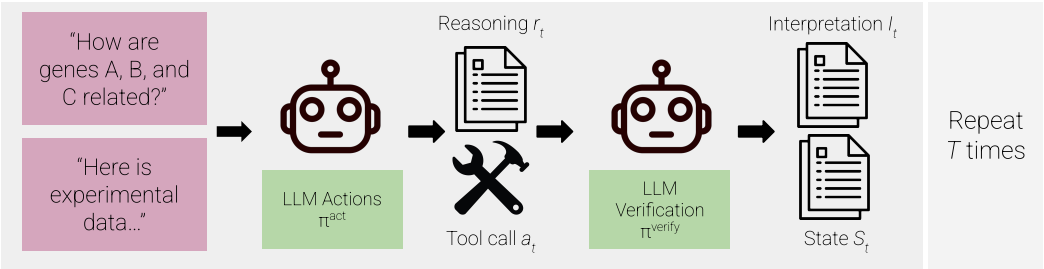


Figure 4.1: The MENTOR-RL agent loop.

1. the current mechanistic claim,
2. the main evidence supporting that claim,
3. any uncertainty, conflict, or alternative explanation,
4. the next question or subgoal the model is trying to resolve.

Structured State

The structured state S_t stores the same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e^{user}),

2. the current relationship status (partially observed group, validated group, multiple groups, or insufficient support) (rel_t),

3. the current predicted group or groups (G_t),

4. the evidence and provenance collected so far (E_t),

5. the current mechanistic labels (y_{mech}),

6. the remaining budget and continuation state ($(T_{\max} - T), z_t$).

4.3.2 Task Outputs and Mechanistic Targets

Final Interpretation

The final outputs of the model are the terminal interpretation I_T and the terminal structured state S_T. Together, these outputs must support four cases: completion of a partially observed mechanistic group, explanation of an already coherent group, refinement of a noisy group, and recognition that no single shared mechanism is supported. The final interpretation states which case is supported by the evidence, summarizes the main evidence for that claim, and links the evidence to the claim through reasoning traces and tool calls. It also records uncertainty or abstention when evidence is incomplete. If the loop terminates with $z_t = \texttt{stop}$, the next subgoal field is left blank. If the loop terminates because the decision budget $T_{\max}$ is exhausted, the interpretation may include a recommended next subgoal to guide further exploration.

Final Structured State

The final structured state is a schema-enforced JSON object. It contains the same components defined in Section **??**, but with finalized values for relationship status, predicted groups, collected evidence, and mechanistic labels. These values are used for reward evaluation and provenance tracking. The final structured state also records whether termination occurred because the model emitted $z_t = \texttt{stop}$ or because the

budget $T_{\max}$ was exhausted. If the budget is exhausted, the remaining budget is 0; if the model stops early, the remaining budget is $T_{\max} - T$.

Supported Task Cases

We support four output cases during training to reflect the variability of experimental evidence that users may provide:

1. partial group completion,
2. mechanism explanation for an already related group,
3. group refinement from a noisy module,
4. no single shared mechanism.

Partial group completion occurs when the user provides a subset of related genes; the model expands this subset into a larger mechanistically related group and explains it. Mechanism explanation occurs when the user provides a complete module; the model explains the mechanism without unnecessary expansion. Group refinement occurs when the user provides a known module with several unrelated noise genes; the agent refines the group so that it contains only mechanistically related genes. Finally, in the no shared mechanism case, the user provides genes that do not support one coherent mechanism, and the model either splits them into multiple groups or returns insufficient support.

Targets for Training and Evaluation

We define several target types for training and evaluation to support dense optimization and comprehensive exploration of biological mechanisms. **Strong targets** consist of groups with experimentally verified membership, such as CORUM complexes. These targets include curated mechanistic annotations such as function, localization, and pathway or category labels. **Derived targets** consist of cases

in which evidence links the query to the predicted group and the final explanation is grounded in retrieved evidence. Derived targets can be further assessed using **Mentor**-style evaluation procedures [122]. We distinguish between these target types to reward recovery of experimentally verified mechanisms while also encouraging the exploration of novel biology.

4.3.3 Data, Resources, and Environment

Our dataset is constructed from a combination of existing internal tool infrastructure, open-source biological resources, and graph exploration environments.

Warm-Start SFT Dataset

We first construct a warm-start SFT subset from the **Mentor-IA** tool stack [130], including gene metadata, gene ontology (GO) annotations, pathway membership, and literature evidence. These data are retrieved via MyGene.info [140] and Europe PMC [**europepmc**]. This warm-start subset covers four task families:

1. entity normalization, such as resolving gene names and synonyms to canonical identifiers,
2. annotation retrieval, such as returning GO terms, localization, and pathway information for a gene or gene set,
3. literature-grounded extraction, such as identifying relevant genes, pathways, or mechanisms from retrieved abstracts, and
4. evidence-to-finding generation, which converts retrieved tool outputs into short, grounded mechanistic statements.

We include tool-enabled (open-book) and tool-disabled (closed-book) questions, with open-book examples emphasized to promote schema-valid tool use and evidence grounding, and closed-book examples used to strengthen biological vocabulary and

lightweight recall. To make this stage reproducible, API responses and database versions will be cached during dataset construction.

To make the warm-start SFT stage concrete, we define a core tool vocabulary derived from the existing **Mentor-IA** infrastructure [130] in Table **??**, with each tool specified by its purpose, input schema, output schema, and training stage. Early SFT uses only this core tool set to teach schema-valid biological tool use before larger graph-exploration operators are introduced during DPO and online RL.

Table 4.1: Tool vocabulary for the **Mentor-RL** agent. Each tool is specified by its purpose, input schema, and the training stage at which it is introduced. SFT uses only the gene metadata retrieval tool; all graph-exploration operators are introduced during DPO and remain available through online RL.

Tool	**Purpose**
`query_mygene`	Retrieve gene metadata, GO annotations, pathway membership, an
`get_neighbors`	Retrieve direct neighbors of a gene in one
`shortest_path`	Compute the shortest path between two genes in a spec
`rwr_multiplex`	Run random walk with restart from one or more seed genes across the full mu
`rwr_monoplex`	Run random walk with restart from one or more seed genes on a single ne
`induce_subgraph`	Extract the subgraph induced by a specified gene set, retur

CORUM-Grounded Training Corpus

After constructing the warm-start SFT dataset, we create a CORUM-grounded training corpus that serves as a human-curated ground-truth dataset for mechanistic discovery. Before task generation, CORUM complexes are split at the complex level into train, validation, and test partitions to prevent leakage across stages. We then define a sampling scheme to produce examples for multiple cases: complex recovery from a small seed set, mechanism explanation from a complete module, complex refinement from a noisy module, and recognition of no single mechanistic relationship.

We define n_g as the number of genes to add or drop during module modification. We then sample a task type from

$$C_{\text{class}} = \{\texttt{recovery}, \texttt{refinement}, \texttt{explanation}, \texttt{none}\},$$

with each class corresponding to one of the supported tasks in Section **??**.

If $C_{\text{class}} = \texttt{none}$, we construct a pseudo-set of unrelated genes as the seed input C^{in}. These genes are sampled from the full CORUM universe under a negative-sampling constraint so that they do not form a single coherent mechanistic group. In this case, we define the target complex as the null set, $C = \varnothing$.

For all other cases, we sample a mechanistically connected module C_{module} from the set of CORUM complexes. If the sampled task is `explanation`, then $C^{\text{in}} = C_{\text{module}} = C$, since the agent should explain the mechanism of an already coherent group without adding or removing genes.

If either `recovery` or `refinement` is sampled, we next sample entities g_i for $i = 1, \ldots, n_g$ without replacement. In the recovery case, genes are sampled from the selected module C_{module} and grouped into G_{drop}. The input set is then defined as

$$C^{\text{in}} = C_{\text{module}} \setminus G_{\text{drop}},$$

while the target remains $C = C_{\text{module}}$.

In the refinement case, noise genes are sampled from the full multiplex universe excluding the selected module, and grouped into G_{add}. The input set is then defined as

$$C^{\text{in}} = C_{\text{module}} \cup G_{\text{add}},$$

while the target remains $C = C_{\text{module}}$. the CORUM-grounded corpus provides supervision for both schema-valid tool use and the terminal outputs defined in Section **??**, including group prediction, relationship status, mechanistic labels, and evidence-grounded explanation.

[t]

Initialize n_g as the number of genes to add or drop Initialize M as global multiplex network Initialize $C_{\text{class}} = \{\texttt{recovery}, \texttt{refinement}, \texttt{explanation}, \texttt{none}\}$ $C_{\text{type}} \sim U(C_{\text{class}})$ $C_{\text{type}} = \texttt{none}$ Sample n_g genes without replacement from multiplex M subject to a negative-sampling constraint $G_{\text{rand}} \leftarrow \{g_i\}_{i=1}^{n_g}$ $C^{\text{in}} \leftarrow G_{\text{rand}}$ $C \leftarrow \varnothing$ $y_{\text{mech}} \leftarrow \varnothing$ $C_{\text{module}} \sim U(\text{CORUM})$ $y_{\text{mech}} \leftarrow \textsc{annotations}(C_{\text{module}})$ $C_{\text{type}} = \texttt{explanation}$ $C^{\text{in}} \leftarrow C_{\text{module}}$ $C \leftarrow C_{\text{module}}$ $C_{\text{type}} = \texttt{recovery}$ Sample n_g genes without replacement from C_{module} $G_{\text{drop}} \leftarrow \{g_i\}_{i=1}^{n_g}$ $C^{\text{in}} \leftarrow C_{\text{module}} \setminus G_{\text{drop}}$ $C \leftarrow C_{\text{module}}$ $C_{\text{type}} = \texttt{refinement}$ Sample n_g genes without replacement from $M \setminus C_{\text{module}}$ $G_{\text{add}} \leftarrow \{g_i\}_{i=1}^{n_g}$ $C^{\text{in}} \leftarrow C_{\text{module}} \cup G_{\text{add}}$ $C \leftarrow C_{\text{module}}$ $(C^{\text{in}}, C, C_{\text{type}}, y_{\text{mech}})$

Graph Environment and Runtime

Each CORUM-derived task instance is paired with a graph environment defined over the same canonical gene universe as the in-house HumanNet multiplex, of which CORUM is a labeled subset. The sampled input set C^{in} initializes the agent's graph anchors, while the hidden target C and associated annotations define the supervisory signal. For negative examples, noise genes are sampled from the full multiplex gene universe rather than only from CORUM, reducing the likelihood of generating an accidentally coherent module.

To interface the graph environment with the training pipeline, we use a native C++ [55] multiplex runtime exposed to Python through pybind11 [56]. Training, rollout, and optimization code are implemented in Python [106], while graph storage and graph operations are implemented in C++ for efficient execution on large multiplexes. Multiplex networks are stored in either custom binary or HDF5 format, depending on scale and interoperability requirements.

The runtime uses a set of optimized multiplex functions developed as part of the **Mentor** [122] and **Mentor-IA** [130] projects. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and

multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the agent during exploration.

All runtime operators return both graph objects and provenance metadata. In addition to nodes, edges, and scores, each returned object records the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers.

4.3.4 Trajectory Generation, Preference Mining, and Mechanistic Summary Construction

Overview

Using the CORUM-grounded corpus and deterministic runtime defined in Sections **??** and **??**, we generate shared-prefix trajectories for SFT and DPO. For each sampled task, the warm-start policy proposes multiple candidate next steps, deterministic execution produces candidate observations and next states. We employ structured branch scoring functions to rank these candidates, and the resulting rankings are converted into shared-prefix preference pairs. The highest-scoring branch at each step is retained as the continuation of the logged trajectory, from which we derive per-step findings and a final mechanistic summary. The trajectory generation process is outlined in Algorithm **??**.

$\pi_\theta^{act}, \pi_\theta^{verify}$, deterministic runtime ε, $T_{\max}, n_{act}, n_{ver}$, pair margin τ. Sample CORUM task $(C^{\text{in}}, C, C_{\text{type}}, y_{mech})$. Sample query/evidence mode ρ $q, e^{\text{user}} \sim Q_{\text{templates}}(\cdot \mid C^{\text{in}}, C, C_{\text{type}}, y_{mech}, \rho)$. $(I_0, S_0) \leftarrow \text{InitState}(q, e^{\text{user}}, C^{\text{in}})$. $\tau^\star \leftarrow [\,]$, $\mathcal{P} \leftarrow [\,]$, $\mathcal{F} \leftarrow [\,]$ $T \leftarrow 0$ $t = 0, 1, \ldots, T_{\max} - 1$ $D_t \leftarrow (q, e^{user}, I_t, S_t)$ $\mathcal{C}_t \leftarrow [\,]$ $i = 0, 1, \ldots, n_{act} - 1$ $(r_{t,i}, a_{t,i}) \sim \pi_\theta^{act}(\cdot \mid D_t)$ $a_{t,i} = \varnothing$ $o_{t,i} \leftarrow \varnothing$ $\text{ValidTool}(a_{t,i}, S_t)$ $o_{t,i} \leftarrow \varepsilon(a_{t,i})$ $o_{t,i} \leftarrow \varnothing$ $j = 0, 1, \ldots, n_{ver} - 1$ $(I'_{t,i,j}, S'_{t,i,j}, z'_{t,i,j}) \sim \pi_\theta^{verify}(\cdot \mid D_t, r_{t,i}, a_{t,i}, o_{t,i})$

$s^{local}_{t,i,j} \leftarrow \text{LocalScore}(D_t, I'_{t,i,j}, S'_{t,i,j}, r_{t,i}, a_{t,i}, o_{t,i}, z'_{t,i,j}; C, C_{\text{type}}, y_{mech})$ Append $c_{t,i,j} = (r_{t,i}, a_{t,i}, o_{t,i}, I'_{t,i,j}, S'_{t,i,j}, z'_{t,i,j})$ to $\mathcal{C}_t$ $\mathcal{P} \leftarrow \mathcal{P} \cup \text{MakePreferencePairs}(\mathcal{C}_t, \tau)$ $(i^\star, j^\star) \leftarrow \text{SelectBranch}(\mathcal{C}_t)$ Append $(I_t, S_t, r_{t,i^\star}, a_{t,i^\star}, o_{t,i^\star}, I'_{t,i^\star,j^\star}, S'_{t,i^\star,j^\star}, z'_{t,i^\star,j^\star})$ to $\tau^\star$ $f_t \leftarrow \text{RenderFinding}(S_t, r_{t,i^\star}, a_{t,i^\star}, o_{t,i^\star}, S'_{t,i^\star,j^\star})$ Append f_t to $\mathcal{F}$ $(I_{t+1}, S_{t+1}) \leftarrow (I'_{t,i^\star,j^\star}, S'_{t,i^\star,j^\star})$ $T \leftarrow t+1$ $z'_{t,i^\star,j^\star} = \texttt{stop}$ **break** $m \leftarrow \text{RenderSummary}(S_T, \mathcal{F})$ $\tau^\star, \mathcal{F}, m, \mathcal{P}$

Shared-prefix Trajectory Synthesis

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ given the current decision context D_t and execute valid tool calls in the deterministic graph runtime. Conditioned on each candidate action and observation, the same policy then samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. The branched sampling scheme is visualized in Figure **??**.

Initialization, Tool Validation, and Query Construction

We define $\text{InitState}(q, e^{\text{user}}, C^{\text{in}})$ as a deterministic function that constructs the initial interpretation I_0 and structured state S_0 from the query, user-provided evidence, and seed gene set. The working interpretation I_0 is populated from a fixed template: the mechanistic claim is empty, the evidence field records only the user-provided inputs, the uncertainty field is blank, and the next subgoal is set to the query q. The structured state S_0 is initialized as follows:

1. the user-provided anchors are set to (q, e^{user}),
2. the relationship status rel_0 is set to `unknown`,
3. the predicted group is initialized to $G_0 = C^{\text{in}}$,
4. the evidence log E_0 is empty,
5. the mechanistic labels y_{mech} are empty,

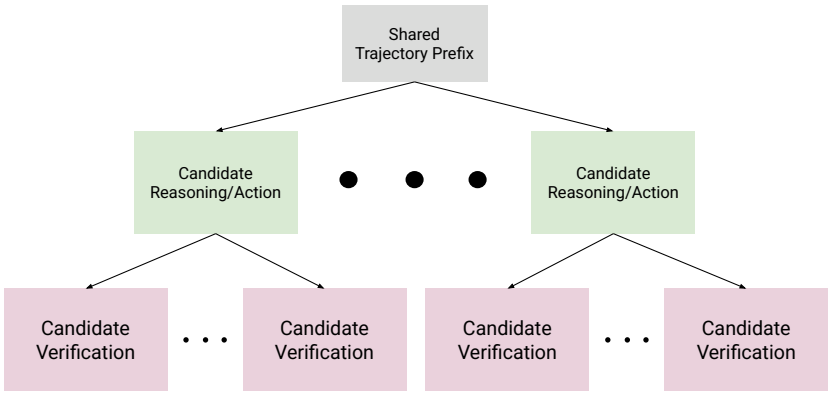


Figure 4.2: The shared prefix generation tree.

6. the remaining budget is $T_{\max}$ and the continuation state is $z_0 = \texttt{continue}$.

The agent does not observe the task type C_{type}, the true relationship status, or the hidden target C; it must infer the appropriate task case through exploration.

We define ValidTool$(a_{t,i}, S_t)$ as a deterministic function that checks whether a proposed tool action $a_{t,i}$ is executable given the current structured state S_t. Validation proceeds in two stages. First, the syntax check verifies that the tool name referenced in $a_{t,i}$ exists in the current tool vocabulary and that all arguments conform to the tool's input schema. Second, the semantic check verifies that basic conditions are satisfied with respect to the graph environment. For example, the function ensures that queried genes exist in the multiplex, requested layers are valid, and required fields in S_t are populated. If either check fails, ValidTool returns false and the observation

$o_{t,i}$ is set to $\varnothing$. Invalid tool calls are counted toward n_{inv} in the efficiency penalty (Equation **??**).

At inference time, users provide a natural language query q alongside optional evidence e^{user}. Queries are unconstrained and may range from broad exploratory requests to specific hypothesis-driven questions. Evidence may include any combination of:

1. a seed gene list,
2. a multiplex graph or subgraph,
3. free-text context such as experimental notes or prior findings, and
4. structured annotations such as GO terms, pathway labels, or expression data from prior analysis.

During training, we simulate this variability by generating diverse queries and evidence configurations from CORUM sample metadata. For each sampled task $(C^{\text{in}}, C, C_{\text{type}}, y_{\text{mech}})$, we sample a query from task-type-specific templates and an evidence mode $\rho \in \{\texttt{minimal}, \texttt{graph}, \texttt{contextual}, \texttt{full}\}$ that controls which evidence types are provided to the agent. Table **??** defines representative query templates and compatible evidence modes for each task type. To promote phrasing diversity, we use an LLM to paraphrase each template query into natural language variants conditioned on the gene names and mechanistic annotations of the sampled complex. All generated queries and evidence configurations are cached during dataset construction for reproducibility.

Branch Ranking, Preference Mining, and Selection

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden CORUM targets. These local scores are used to rank same-prefix candidates, mine DPO preference pairs, and select the

Table 4.2: Representative query templates and compatible evidence modes by task type. Evidence modes: `minimal` (seed gene list), `graph` (seed gene list + multiplex subgraph), `contextual` (seed gene list + free-text context), `full` (all evidence types including structured annotations).

Task type	Example query	Evidence modes
`recovery`	"What other genes are functionally related to {seed genes}?"	all
`refinement`	"Which of these genes do not belong together?"	all
`explanation`	"What mechanism connects {seed genes}?"	all
`none`	"Are these genes mechanistically related?"	all

continuation branch for the logged trajectory. We defer the formal score definitions to Section **??**.

4.3.5 Preference Scoring and Reward Design

Design Principles and Scoreable Objects

We define all training-time scores over structured state transitions, deterministic runtime metadata, and hidden CORUM-derived targets rather than textual reasoning. This design choice keeps scoring reproducible, machine-parseable, and scalable, while avoiding dependence on human judges or semantic evaluation of natural-language trajectories. We decompose all scores into four components: schema compliance, complex membership, mechanistic label accuracy, and efficiency. Additionally, we distinguish between local scores, which measure per-step improvement for shared-prefix comparison, and terminal rewards, which include absolute recovery for trajectory-level evaluation.

Deterministic Local Branch Scoring for DPO

We define local branch scoring for DPO as

$$s_{t,i,j}^{local} = w_s s_{t,i,j}^{schema} + w_c \Delta s_{t,i,j}^{complex} + w_m \Delta s_{t,i,j}^{mech} - w_e p_{t,i,j}^{eff}, \tag{4.1}$$

which calculates a composite score from schema compliance, change in complex membership, change in mechanism accuracy, and an efficiency penalty. Individual scoring contributions are controlled with scalar weights w, which will be tuned on the validation partition. This local score is used only for same-prefix branch comparison and continuation selection during DPO trajectory synthesis.

We define $s_{t,i,j}^{schema}$ as a binary indicator that equals 1 if the candidate's structured state and interpretation updates are schema valid, and 0 otherwise. Specifically, schema compliance is defined by a properly-formed JSON with all required fields present and tool calls referencing valid tool names and arguments. We formalize this term as

$$s_{t,i,j}^{schema} = \mathbf{1}(\text{SchemaValid}(c_{t,i,j})). \tag{4.2}$$

We use $\Delta s_{complex}$ to measure the change in complex membership accuracy from step t to $t+1$. We define this metric as a composite:

$$\begin{aligned}\Delta s_{t,i,j}^{complex} = \alpha(J(G_{t+1,i,j}, C) - J(G_{t,i,j}, C)) + \\ \beta(P(G_{t+1,i,j}, C) - P(G_{t,i,j}, C)) + \gamma(R(G_{t+1,i,j}, C) - R(G_{t,i,j}, C)),\end{aligned} \tag{4.3}$$

where J, P, and R denote Jaccard, precision, and recall respectively, and α, β and γ are weighting coefficients. Weighted terms enable prioritization of different metrics for different task types. For example, recall can be upweighted for recovery tasks, rewarding found members, while precision can be upweighted for refinement tasks, penalizing retained noise.

We define Δs_{mech} as the change in mechanistic label accuracy, defined as the difference in the proportion of correctly predicted labels out of total labels between steps t and $t+1$. We formalize this score as

$$\Delta s_{t,i,j}^{mech} = (y_{t+1,i,j}^{true} / y_{t+1,i,j}^{total}) - (y_{t,i,j}^{true} / y_{t,i,j}^{total}). \tag{4.4}$$

If no mechanistic labels have been predicted at step t, we set $\Delta s_{t,i,j}^{mech} = 0$.

Finally, p_{eff} is a small penalty designed to discourage two behaviors: excessively long trajectories and invalid tool calling. Long trajectories are penalized via the term $t/T_{\max}$, while we count invalid tool calls (those that are schema-invalid, exact duplicates, or return empty observations) with n_{inv}, penalizing them proportionally to the total number of tool calls: (n_{inv}/n_{total}). We formalize this scoring term as

$$p^{eff}_{t,i,j} = \lambda_1(t/T_{\max}) + \lambda_2(n_{inv}/n_{total}). \tag{4.5}$$

Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$R_T = w_s^T s^T_{schema} + w^T_{abs} s^T_{complex} + w_c^T \Delta s^T_{complex} + w^T_{mabs} s^T_{mech} + w^T_m \Delta s^T_{mech} - w_e^T p^T_{eff}. \tag{4.6}$$

While all terms included in local scoring appear in the terminal reward, they are modified to capture global behavior. Additionally, two new rewards are included to capture absolute recovery. Absolute terms appear in the terminal score but not the local score. Because local scoring is evaluated on examples that share the same starting state S_t, absolute levels cancel, and only relative metrics matter. Conversely, the terminal reward is evaluating complete trajectories, which may vary in starting difficulty and length, so absolute metrics capture global problem-solving ability.

We define the absolute complex membership score as the composite Jaccard, precision, and recall evaluated on the final predicted group G_T against the hidden target C:

$$s^T_{complex} = \alpha J(G_T, C) + \beta P(G_T, C) + \gamma R(G_T, C), \tag{4.7}$$

using the same weighting coefficients α, β, γ defined in Equation **??**. The cumulative complex delta is the difference between the final and initial composite scores:

$$\Delta s^T_{complex} = s^T_{complex} - s^0_{complex}, \tag{4.8}$$

where $s^0_{complex} = \alpha J(G_0, C) + \beta P(G_0, C) + \gamma R(G_0, C)$ is the composite score at initialization.

The absolute mechanistic label accuracy is the final proportion of correctly predicted labels:

$$s^T_{mech} = y^{true}_T / y^{total}_T. \tag{4.9}$$

The cumulative mechanistic delta is the corresponding difference:

$$\Delta s^T_{mech} = s^T_{mech} - s^0_{mech}. \tag{4.10}$$

If no mechanistic labels have been predicted at termination, we set $s^T_{mech} = 0$ and $\Delta s^T_{mech} = 0$.

The terminal schema score checks that the final structured state is schema-valid and that the termination mode is properly recorded:

$$s^T_{schema} = \mathbf{1}(\text{SchemaValid}(S_T) \wedge \text{TerminationRecorded}(S_T)). \tag{4.11}$$

Finally, the terminal efficiency penalty aggregates trajectory length and invalid tool use across all steps:

$$p^T_{eff} = \lambda_1 \left(\frac{T}{T_{\max}} \right) + \lambda_2 \left(\frac{\sum_{t=0}^{T-1} n^t_{inv}}{\sum_{t=0}^{T-1} n^t_{total}} \right). \tag{4.12}$$

Preference Pair Construction, Score Normalization, and Filtering

At each decision point t, we create DPO preference tuples of the form (x_t, c_t^+, c_t^-) from our candidate set $\mathcal{C}_t = \{c_{t,i,j}\}$. We define $x_t = (q, e^{\text{user}}, I_t, S_t)$ as the shared-prefix context and $c_{t,i,j}$ as a sample interpretation continuation from Algorithm **??**. Each continuation contains the model-generated reasoning step, tool action, verifier update, and continuation decision.

Before constructing pairs, we normalize scores to ensure that the selection margin has a consistent meaning across task types and trajectory lengths. We apply min-max normalization within each candidate pool $\mathcal{C}_t$, rescaling the composite local scores to $[0, 1]$. We denote the normalized score of candidate $c_{t,i,j}$ as $\bar{s}_{t,i,j}^{\text{local}}$.

To create paired examples, we first select the preferred candidate c_t^+ as the candidate that produces the highest normalized score $\bar{s}_{t,i,j}^{\text{local}}$. We then define a selection margin τ and drop all candidates for which

$$\bar{s}_{t,i^*,j^*}^{\text{local}} - \bar{s}_{t,i,j}^{\text{local}} < \tau,$$

where (i^*, j^*) indexes the preferred candidate. This prevents the construction of pairs that are near ties. The remaining candidates form the eligible dispreferred pool $\mathcal{C}_t^-$.

From $\mathcal{C}_t^-$, we draw dispreferred examples across three difficulty bins, $d \in \{\texttt{easy}, \texttt{medium}, \texttt{hard}\}$. We construct `easy` pairs by selecting the lowest-scoring candidate in $\mathcal{C}_t^-$, `medium` pairs by selecting the candidate with the median score, and `hard` pairs by selecting the highest-scoring candidate in $\mathcal{C}_t^-$.

We store each preference pair (x_t, c_t^+, c_t^-) along with its difficulty bin d, task type C_{type}, decision step t, raw and normalized scores, and the score margin in the preference corpus $\mathcal{P}$. After trajectory generation is complete, we rebalance $\mathcal{P}$ across task types and difficulty bins by downsampling overrepresented categories to ensure that the training distribution is not dominated by task types that produce

disproportionately many informative pairs. The curriculum strategy described in Section **??** controls which difficulty bins are included at each training stage.

Mechanistic Rendering and Training Outputs

Each selected trajectory is logged as a sequence of turn records, from which we render short per-step findings and a final mechanistic summary. These artifacts are then converted into SFT trajectory examples, evidence-to-finding records, and DPO preference pairs for the downstream training pipeline. Additionally, scored trajectories can be utilized for policy gradient optimization, such as PPO or GRPO.

4.3.6 Training Pipeline

We propose a training pipeline that closely aligns with standard model post-training pipelines. First, we utilize supervised fine-tuning for general biological knowledge and tool calling. We continue training with DPO on stepwise interpretation tree comparisons. Finally, we promote novel exploration over long-horizon tasks with policy gradient optimization.

Warm-Start SFT

We first train our policy model with warm-start SFT using the closed-book dataset outlined in Section **??**. We will train our policy model on OLCF's Frontier Supercomputer [4], utilizing slurm [146] for job scheduling, huggingface and TRL [139] for weight updating, DeepSpeed ZeRO-3 for model parallelism [100], Low-Rank Adaptation for parameter-efficient fine-tuning [49], Weights and Biases [9] for tracking convergence, and Kimi-K2 [124] as our base policy. We will utilize early stopping checkpointing to prevent model overtraining.

Tool-Call and Evidence-to-Finding SFT

We continue the supervised fine-tuning stage by adding open-book tool calling examples and evidence-to-finding examples to our data mixture. We initialize our policy from the warm-start SFT checkpoint with the lowest loss, and we utilize the same pipeline for model training.

DPO Training

After training our model via SFT, we will utilize its learned domain knowledge and tool calling capabilities to perform DPO training. We will utilize examples generated with the trajectory construction methods specified in **??** to create paired preference data. Trajectories will be scored using the functions defined in Section **??**; the highest-scoring trajectory will be used as the preferred candidate, while either the lowest-scoring or median-scoring trajectories will be utilized as the dispreferred candidate. Model weights will be optimized using the DPO objective function [98]:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

Additionally, during training, we will generate trajectories with new, trained model checkpoints to improve our model's generated preference data. This will allow us to continue making model improvements even after scoring is optimized on our initial dataset.

Online RL for Long-Horizon Exploration

After training with DPO, we will implement a final GRPO stage for long-horizon exploration. Unlike DPO, GRPO requires no annotated trajectories; rather, optimization is done by calculating the advantage of an output relative to the rollout group $\{o_1, o_2, \ldots, o_G\}$ using the function

$$\mathcal{J}_{GRPO} = \mathbb{E}[q \sim P(Q), \{y_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(Y \mid q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{\mid y_i \mid}\sum_{t=1}^{|y_i|}\left\{\min\Big[\frac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})}\hat{A}_{i,t}, \text{clip}\Big(\frac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t} \mid q, y_{i,<t})}, 1-\varepsilon, 1+\varepsilon\Big)\hat{A}_{i,t}\Big] - \beta\mathbb{D}_{KL}[\pi_\theta \mid\mid \pi_{ref}]\right\},$$

where

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{R_i^T - \text{mean}(R^T)}{\text{std}(R^T)}$$

By foregoing structured, annotated preference pairs, GRPO promotes novel exploration for problem solving. We aim to utilize this policy gradient method to prevent mode collapse in exploration trajectories and reinforce full coverage of the exploration space. The reference policy π_{ref} is fixed at the DPO checkpoint that initializes GRPO and is not updated during online rollouts, so the KL term anchors exploration to a stable reference rather than drifting with the trained policy.

4.3.7 Curriculum and Optimization Strategy

We will employ a training curriculum throughout the optimization process, scaling problem difficulty with model performance.

During the SFT stage, we scale difficulty by moving from closed-book biological questions to open-book biological questions that require tool use. This promotes learning biological understanding first, and then moving to structured, schema-respecting tool calling tasks second.

After training a joint closed- and open-book SFT checkpoint, we scale difficulty for DPO pairs by training on increasingly difficult examples, based on difficulty bins defined in Section **??**. We begin DPO training by limiting preference pairs to $d =$ `easy` examples, then adding `medium` and `hard` examples as training saturates for each

difficulty. After DPO training converges on `hard` preference pairs, we transition to online RL to promote exploration beyond the behaviors captured in logged trajectory outputs.

We scale long-horizon reinforcement learning difficulty with several strategies. First, we modulate the trajectory length $T_{\max}$, starting with shorter exploration budgets and gradually increasing exploration length for problems that require multi-hop exploration. Additionally, we modulate complex size and difficulty, beginning training on small complexes and shifting to larger ones as rewards saturate. Finally, we scale the noise-to-signal ratio n_g, beginning with fewer added and dropped genes and progressively adding noise to examples to make recovery more difficult.

4.3.8 Evaluation and Benchmarks

CORUM Holdout Evaluation

We will use several metrics to evaluate the effectiveness of our model. Each metric will be calculated by parsing the model's terminal structured state S_T, allowing for scalable evaluation of our checkpoints. All evaluations will be conducted on a held out group of CORUM complex examples.

We will report four metrics for evaluation: Jaccard index J, precision P, recall R, and F_1 score on predicted CORUM complexes C^{pred} vs. ground truth complexes C. Jaccard index is defined as

$$J(C^{pred}, C) = \frac{\mid C^{pred} \cap C \mid}{\mid C^{pred} \cup C \mid},$$

penalizing both missing and extra entities. Precision is defined as

$$P = \frac{TP}{TP + FP},$$

measuring the proportion of true positive entities compared to all predicted positives. Recall is defined as

$$R = \frac{TP}{TP + FN},$$

measuring the proportion of true positive predictions to the sum of true positives and false negatives. The F1 score is defined as the harmonic mean of precision and recall,

$$F1 = \frac{2}{\frac{1}{P} + \frac{1}{R}},$$

providing a balanced view of precision and recall to measure general performance. In addition to these complex recovery metrics, we will measure mechanistic label accuracy, defined in Equation **??**, to ensure that each recovered gene is correctly annotated.

We will report the weighted composite reward R_T, defined in Section **??**, to directly compare training reward magnitude on validation and holdout splits. If the task type is `none`, meaning $C = \varnothing$, we will parse the S_T for the predicted relationship rel_T. Along with computing global accuracy, we will distinguish between both error directions, analyzing when the model claims no mechanism exists for truly related gene sets and when the model fabricates a mechanism for truly unrelated entities.

Furthermore, we will report efficiency metrics, such as mean trajectory length, budget utilization, and invalid tool call rate. This allows for comparison between holdout efficiency and validation efficiency metrics computed in Equation **??**.

In addition to general evaluation, we will stratify our evaluation across several axes: complex size, noise, and task type. We will divide complex size and noise into three discrete bins each for comparative evaluation. Reporting stratified metrics allows us to analyze how **Mentor-RL** performs as task difficulty increases and task type changes.

Novel-Domain Evaluation

In addition to empirical evaluation using CORUM complexes as ground truth, we will utilize MENTOR to conduct evaluations on novel experimental data. Current versions of the MENTOR pipeline significance measures for gene modules yield a ground truth for complexes outside the mammalian protein realm. We will use previously derived experimental data, such as brain multiplex data, as our ground truth. Because these examples are derived from MENTOR results on real experimental data, this process will allow us to assess the performance of **Mentor-RL** on examples outside the curated CORUM universe using the same evaluation techniques described in Section **??**. As such, we will gain a better understanding of the strengths and limitations of **Mentor-RL** on out-of-distribution, real-world data, while still having expert- and algorithm-verified ground truth examples to compare to.

Blind Comparative Evaluation

In conjunction with empirical validation, which measures model accuracy, we will conduct blind comparative evaluations to measure expert domain scientists' model preferences. First, we will perform an inter-model preference test, supplying interpretation outputs from **Mentor-RL**, GPT-5 [118], gpt-oss-120b [89], LLaMA 4 [2], Kimi K2 [124], and Nemotron 3 Super [87]. We will provide between 5 and 15 domain experts with paired samples containing **Mentor-RL** outputs and outputs from another randomly selected model. We will then ask scientists to indicate their preferred interpretation, and with this information, we will calculate the proportion of **Mentor-RL** outputs that are preferred compared to other models. To support interpretation of these proportions, we will overlap a subset of evaluation pairs across raters and report inter-expert rating reliability.

After conducting inter-model preference testing, we will perform intra-model preference testing to compare preferences for different model checkpoints. Using the same protocol described for inter-model comparative evaluation, we will compare the

base, SFT, DPO, and GRPO-trained **Mentor-RL** checkpoints. Blind comparative testing will allow us to evaluate model characteristics such as style, tone, and concision that are not easily evaluable with empirical metrics.

Efficiency and Ablations

To ensure that each component of our methodology contributes to an improvement in mechanistic interpretation, we will ablate both training stages and design elements. All ablations will be evaluated using the holdout metrics defined in Section **??**.

We will ablate phases of our training pipeline to confirm that each stage improves over the previous checkpoints. First, we will test our SFT-only checkpoint compared to the SFT- and DPO-trained checkpoint to ensure that preference optimization yields interpretation gains. Next, we will compare the DPO checkpoint to the full training pipeline to determine whether long-horizon RL training leads to improvement over local preference training. Finally, we will examine the effects of training with only closed-book SFT data vs. a mixed closed- and open-book training set to evaluate whether tool-calling examples in SFT lead to better DPO optimization.

Along with testing various training stages, we will ablate architectural and reward decisions. First, we will ablate the policy's verifier mode, removing it to test whether self-verification is useful for producing better results. Additionally, we will remove the efficiency penalty p^{eff} from the reward to test whether **Mentor-RL** can learn reasonable trajectory lengths without regularization. Finally, we will ablate difficulty bins in our training curriculum, training on all difficulty bins simultaneously rather than staging them progressively. This ablation will allow us to examine whether the model can learn from hard examples without curriculum staging.

4.4 Expected Results and Deliverables

4.4.1 Expected Empirical Results

We expect **Mentor-RL** to recover held-out CORUM complexes with a composite F_1 competitive with frontier models at substantially lower per-query inference cost while improving monotonically across the training pipeline. Each of the SFT, DPO, and GRPO checkpoints should outperform the preceding one on complex recovery and mechanistic label accuracy. Under the stratified evaluation defined in Section **??**, we expect the largest gains on `recovery` and `refinement` tasks, where deterministic graph operators provide dense supervisory signals, and smaller but nontrivial gains on `explanation` and `none` tasks, where reward is driven by mechanistic labeling and abstention rather than set-level recovery. On the novel-domain evaluation against MENTOR-derived modules, we expect a performance gap relative to CORUM holdout, reflecting distributional shift, but with the gap narrowing as curriculum difficulty and noise levels increase during training.

4.4.2 Expected Behavioral Results

Beyond empirical accuracy, we expect three behavioral outcomes. First, the act-verify loop should reduce schema-invalid tool calls and unsupported mechanistic claims relative to a verifier-ablated baseline, measured by invalid tool call rate and ungrounded-claim rate on the holdout split. Second, the efficiency penalty should produce shorter average trajectories without loss of recovery accuracy relative to a penalty-ablated baseline. Third, blind comparative evaluation should show that expert scientists prefer **Mentor-RL** outputs to base-policy outputs at a significantly higher rate, and preferences relative to frontier models should improve across successive training stages.

4.4.3 Deliverables

We will release, under licenses compatible with the underlying data sources, the following artifacts:

1. the CORUM-grounded training environment, including the C++/pybind11 multiplex runtime, tool vocabulary, and deterministic scoring functions,

2. train, validation, and test split indices at the complex level, together with the code required to reconstruct the raw training corpus from CORUM and cached API responses,

3. trained **Mentor-RL** checkpoints for each stage of the pipeline (warm-start SFT, joint closed- and open-book SFT, DPO, and GRPO),

4. the evaluation harness, including CORUM holdout benchmarks, novel-domain MENTOR-derived benchmarks, and blind comparative evaluation protocols, and

5. training and generation code, curriculum configurations, and logged trajectories to support reproducible re-training and ablation.

4.4.4 Broader Impact

By releasing a deterministically-scored, open-source mechanistic exploration agent together with its training environment, we aim to lower the cost of biological interpretation research and establish a reproducible benchmark that does not require proprietary model access. The CORUM-grounded scoring framework is domain-specific, but the underlying pattern, deterministic structured-state rewards over a tool-using policy, is transferable to other scientific settings with curated ground truth.

4.5 Discussion and Conclusion

4.5.1 CORUM Licensing and Data Redistribution

Because the CORUM dataset is restricted to academic, non-commercial use, and redistribution of the raw data is not permitted under standard terms, our model releases will not include the raw dataset. To preserve reproducibility, we will release train, validation, and test split indices at the complex level together with the code needed to reconstruct the raw training corpus directly from CORUM. Cached API responses from databases such as MyGene.info and Europe PMC will be distributed under their respective licenses.

4.5.2 Reproducibility Without Proprietary Judges

The training pipeline for **Mentor-RL** foregoes the use of proprietary judge models to ensure reproducibility and compliance with terms of use. Rewards are computed deterministically over structured state transitions and CORUM-derived ground truth rather than by semantic evaluation from a frontier model. Trajectories are generated either by sampling the policy under training or by deterministic templating, not by proxy labeling through a closed API. One exception is query paraphrasing during dataset construction, for which we will use an open-source model with a pinned version so that the paraphrasing step can be reproduced without API access. Despite avoiding proprietary models during training, we will still compare **Mentor-RL** to them through blind comparative evaluation, detailed in Section **??**.

4.5.3 Limitations

Several limitations constrain the scope of the claims we can make with this work.

Ground-truth coverage. CORUM covers mammalian protein complexes and does not capture the full space of mechanistic biological relationships such as regulatory circuits, signaling cascades, or cross-compartment interactions. A model trained to

recover CORUM complexes may underperform on relationships that are mechanistic but non-complex in nature. While our novel-domain evaluation using MENTOR-derived modules partially addresses this, the gap is not eliminated.

Reward-hacking risk. The composite complex-membership score rewards moves toward the hidden target C. Since C is never observed directly, the agent cannot short-circuit it, but shortcut policies are still possible. For example, a policy that always returns the top-k random-walk-with-restart output from the seed set will recover a substantial fraction of CORUM complexes without genuine multi-step reasoning. The efficiency penalty does not counteract this behavior. We will monitor tool-use entropy and trajectory diversity during training and, if shortcut behavior emerges, introduce adversarially constructed complexes or a diversity bonus as mitigations.

Shared-weight verifier. The actor and verifier are two prompting modes of the same policy, so DPO and RL updates on one mode also update the other. This introduces a risk of verifier collapse, in which the verifier learns to endorse the actor's proposals regardless of quality. We will monitor the verifier disagreement rate and, if collapse is observed, consider separate LoRA adapters per mode or a consistency regularizer between modes.

Task difficulty imbalance. The four task types are sampled uniformly during corpus construction, but they are not of equal difficulty; `explanation` tasks require no graph modification, while `refinement` tasks require the verifier to remove members without a tool that directly marks membership. We treat the per-task sampling weights as a hyperparameter and expect to tune them during pilot training.

Split-level leakage. CORUM complexes share subunits across entries; therefore, a complex-level split does not by itself prevent leakage through shared subunits. We will report overlap statistics between train and held-out splits and, if overlap is material, adopt a subunit-aware split strategy.

Expert evaluation scope. The blind comparative evaluation relies on 5 to 15 domain experts making pairwise preference judgments. This sample size supports descriptive

comparison but not tight confidence intervals on preference rates, and expert rating reliability will be reported alongside preference proportions.

4.5.4 Conclusion

We have presented **Mentor-RL**, a proposed open-source reasoning agent for biological mechanistic interpretation, trained through a staged pipeline of supervised fine-tuning, direct preference optimization over shared-prefix trajectory branches, and policy gradient optimization for long-horizon exploration. The training environment grounds rewards in deterministic graph operators and CORUM-derived targets, removing proprietary judges from the loop and exposing every optimization signal as a reproducible, structured quantity. We view **Mentor-RL** as one instance of a broader methodological pattern: domain-specialized scientific agents whose behavior is shaped by deterministic structured-state rewards over curated ground truth, rather than by semantic evaluation from frontier models. If successful, this pattern offers a route toward scientific AI systems that are cheap to deploy, auditable in their reasoning, and reproducible across research groups without centralized model access.

I Agent Loop Algorithm

[t]

Initialize I_0, S_0 from query q and user evidence e^{user} $T \leftarrow 0$ $t = 0, 1, \ldots, T_{\max} - 1$ $D_t \leftarrow (q, e^{\text{user}}, I_t, S_t)$ $(r_t, a_t) \sim \pi_\theta^{\text{act}}(\cdot \mid D_t)$ $a_t \neq \varnothing$ $o_t \leftarrow \varepsilon(a_t)$ $o_t \leftarrow \varnothing$ $(I_{t+1}, S_{t+1}, z_t) \sim \pi_\theta^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t)$ $T \leftarrow t + 1$ $z_t =$ `stop` **break** final interpretation I_T and structured state S_T

II CORUM Corpus Creation Algorithm

III Trajectory Generation Algorithm

Bibliography

[1] Texas Tech Davis College of Agricultural Sciences Natural Resources. *Research Farm at New Deal*. 2025. URL: https://www.depts.ttu.edu/agriculturalsciences/About/facilities/maps/resFarm.php.

[2] Redacted by arXiv. *The Llama 4 Herd: Architecture, Training, Evaluation, and Deployment Notes*. 2026. arXiv: 2601.11659 [cs.SE]. URL: https://arxiv.org/abs/2601.11659.

[3] Akari Asai et al. "Self-rag: Learning to retrieve, generate, and critique through self-reflection". In: *The Twelfth International Conference on Learning Representations*. 2023.

[4] Scott Atchley et al. "Frontier: Exploring Exascale". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '23. Denver, CO, USA: Association for Computing Machinery, 2023. ISBN: 9798400701092. DOI: 10.1145/3581784.3607089. URL: https://doi.org/10.1145/3581784.3607089.

[5] Mohammad Gheshlaghi Azar et al. "A general theoretical paradigm to understand learning from human preferences". In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2024, pp. 4447–4455.

[6] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. "Network medicine: a network-based approach to human disease". In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.

[7] Albert-Laszlo Barabasi and Zoltan N Oltvai. "Network biology: understanding the cell's functional organization". In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.

[8] Sumanta Basu et al. "Iterative random forests to discover predictive and stable high-order interactions". In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.

[9] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: https://www.wandb.com/.

[10] Vincent D Blondel et al. "Fast unfolding of communities in large networks". In: *Journal of statistical mechanics: theory and experiment* 2008.10 (2008), P10008.

[11] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[12] G. Bradski. "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools* (2000).

[13] Nicolas Carion et al. "Sam 3: Segment anything with concepts". In: *arXiv preprint arXiv:2511.16719* (2025).

[14] Liang-Chieh Chen et al. "Encoder-decoder with atrous separable convolution for semantic image segmentation". In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 801–818.

[15] Yiqun Chen and James Zou. "Simple and effective embedding model for single-cell biology built from ChatGPT". In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.

[16] Bowen Cheng et al. "Mask2former for video instance segmentation". In: *arXiv preprint arXiv:2112.10764* (2021).

[17] Paul F Christiano et al. "Deep reinforcement learning from human preferences". In: *Advances in neural information processing systems* 30 (2017).

[18] Ashley Cliff et al. “A high-performance computing implementation of iterative random forest for the creation of predictive expression networks”. In: *Genes* 10.12 (2019), p. 996.

[19] Lenore Cowen et al. “Network propagation: a universal amplifier of genetic associations”. In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.

[20] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.

[21] Hugo Dalla-Torre et al. “Nucleotide Transformer: building and evaluating robust foundation models for human genomics”. In: *Nature Methods* 22.2 (2025), pp. 287–297.

[22] Manlio De Domenico et al. “Mathematical formulation of multilayer networks”. In: *Physical Review X* 3.4 (2013), p. 041022.

[23] Jia Deng et al. “Imagenet: A large-scale hierarchical image database”. In: *2009 IEEE conference on computer vision and pattern recognition*. Ieee. 2009, pp. 248–255.

[24] Jacob Devlin et al. “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[25] Kieran Elmes et al. “SNVformer: An Attention-based Deep Neural Network for GWAS Data”. In: *bioRxiv* (2022), pp. 2022–07.

[26] Mark Everingham et al. “The pascal visual object classes (voc) challenge”. In: *International journal of computer vision* 88 (2010), pp. 303–338.

[27] Li Fei-Fei, Rob Fergus, and Pietro Perona. “Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories”. In: *2004 conference on computer vision and pattern recognition workshop*. IEEE. 2004, pp. 178–178.

[28] Christiane Fellbaum. *WordNet: An electronic lexical database*. MIT press, 1998.

[29] Kunihiko Fukushima. "Cognitron: A self-organizing multilayered neural network". In: *Biological cybernetics* 20.3 (1975), pp. 121–136.

[30] Kunihiko Fukushima. "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position". In: *Biological cybernetics* 36.4 (1980), pp. 193–202.

[31] Luyu Gao et al. "Pal: Program-aided language models". In: *International conference on machine learning*. PMLR. 2023, pp. 10764–10799.

[32] Margarita Garcia-Hernandez et al. "TAIR: a resource for integrated Arabidopsis data". In: *Functional & integrative genomics* 2.6 (2002), pp. 239–253.

[33] Marc Gillespie et al. "The reactome pathway knowledgebase 2022". In: *Nucleic acids research* 50.D1 (2022), pp. D687–D692.

[34] Ross Girshick. "Fast r-cnn". In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1440–1448.

[35] Madalina Giurgiu et al. "CORUM: the comprehensive resource of mammalian protein complexes—2019". In: *Nucleic acids research* 47.D1 (2019), pp. D559–D563.

[36] Xavier Glorot and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2010, pp. 249–256.

[37] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. "Deep sparse rectifier neural networks". In: *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2011, pp. 315–323.

[38] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[39] Daya Guo et al. "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning". In: *arXiv preprint arXiv:2501.12948* (2025).

[40] Leland H Hartwell et al. "From molecular to modular cell biology". In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.

[41] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[42] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[43] Kaiming He et al. "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification". In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1026–1034.

[44] Kaiming He et al. "Mask r-cnn". In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 2961–2969.

[45] Donald O Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: John Wiley & Sons, 1949.

[46] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. "A fast learning algorithm for deep belief nets". In: *Neural computation* 18.7 (2006), pp. 1527–1554.

[47] Jiwoo Hong, Noah Lee, and James Thorne. "Orpo: Monolithic preference optimization without reference model". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 11170–11189.

[48] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators". In: *Neural networks* 2.5 (1989), pp. 359–366.

[49] Edward J Hu et al. “Lora: Low-rank adaptation of large language models.” In: *Iclr* 1.2 (2022), p. 3.

[50] David H Hubel and Torsten N Wiesel. “Receptive fields of single neurones in the cat’s striate cortex”. In: *The Journal of Physiology* 148.3 (1959), pp. 574–591.

[51] David H Hubel and Torsten N Wiesel. “Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex”. In: *The Journal of physiology* 160.1 (1962), pp. 106–154.

[52] Trey Ideker and Nevan J Krogan. “Differential network biology”. In: *Molecular systems biology* 8 (2012), p. 565.

[53] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. pmlr. 2015, pp. 448–456.

[54] Fabian Isensee et al. “nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation”. In: *Nature methods* 18.2 (2021), pp. 203–211.

[55] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: http://www.iso.org.

[56] Wenzel Jakob, Jason Rhinelander, and Dean Moldovan. *pybind11 – Seamless operability between C++11 and Python*. https://github.com/pybind/pybind11. 2017.

[57] Yanrong Ji et al. “DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome”. In: *Bioinformatics* 37.15 (2021), pp. 2112–2120.

[58] Yangqing Jia et al. “Caffe: Convolutional Architecture for Fast Feature Embedding”. In: *arXiv preprint arXiv:1408.5093* (2014).

[59] Jens Kattge et al. "TRY–a global database of plant traits". In: *Global change biology* 17.9 (2011), pp. 2905–2935.

[60] Diederik P Kingma. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[61] Thomas N Kipf and Max Welling. "Semi-supervised classification with graph convolutional networks". In: *arXiv preprint arXiv:1609.02907* (2016).

[62] Alexander Kirillov et al. "Segment anything". In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2023, pp. 4015–4026.

[63] Sebastian Köhler et al. "Walking the interactome for prioritization of candidate disease genes". In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.

[64] Takeshi Kojima et al. "Large language models are zero-shot reasoners". In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.

[65] Alex Krizhevsky. "Learning Multiple Layers of Features from Tiny Images". In: (2009), pp. 32–33. URL: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[66] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in neural information processing systems* 25 (2012).

[67] John Lagergren et al. *Few-Shot Learning Enables Population-Scale Analysis of Leaf Traits in Populus trichocarpa*. 2023. arXiv: 2301.10351 [cs.CV].

[68] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.

[69] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.

[70] Yann LeCun, Ido Kanter, and Solla A Solla. "Efficient BackProp". In: *Neural Networks: Tricks of the Trade*. Originally published in 1991 as 'Eigenvalues of covariance matrices: Application to neural-network learning'. Springer, 2012, pp. 9–50.

[71] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[72] Yann LeCun et al. "Handwritten digit recognition with a back-propagation network". In: *Advances in neural information processing systems* 2 (1989).

[73] Tsung-Yi Lin et al. "Microsoft coco: Common objects in context". In: *European conference on computer vision*. Springer. 2014, pp. 740–755.

[74] Seppo Linnainmaa. "The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors". PhD thesis. Master's Thesis (in Finnish), Univ. Helsinki, 1970.

[75] Ilya Loshchilov and Frank Hutter. "Decoupled weight decay regularization". In: *arXiv preprint arXiv:1711.05101* (2017).

[76] Pan Lu et al. "Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome". In: *bioRxiv* (2026). DOI: 10.64898/2026.02.27.708412. eprint: https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.full.pdf. URL: https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.

[77] Aman Madaan et al. "Self-refine: Iterative refinement with self-feedback". In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.

[78] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[79] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The bulletin of mathematical biophysics* 5.4 (1943), pp. 115–133.

[80] Jörg Menche et al. "Uncovering disease-disease relationships through the incomplete interactome". In: *Science* 347.6224 (2015), p. 1257601.

[81] Marija Milacic et al. "The reactome pathway knowledgebase 2024". en. In: *Nucleic Acids Res.* 52.D1 (Jan. 2024), pp. D672–D678.

[82] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969.

[83] Peter J Mucha et al. "Community structure in time-dependent, multiscale, and multiplex networks". In: *science* 328.5980 (2010), pp. 876–878.

[84] Maria L. Murgia et al. "A Comprehensive Phenotypic Investigation of the "Pod-Shattering Syndrome" in Common Bean". In: *Frontiers in Plant Science* 8 (2017). ISSN: 1664-462X. DOI: 10.3389/fpls.2017.00251. URL: https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2017.00251.

[85] Reiichiro Nakano et al. "Webgpt: Browser-assisted question-answering with human feedback". In: *arXiv preprint arXiv:2112.09332* (2021).

[86] Mark EJ Newman and Michelle Girvan. "Finding and evaluating community structure in networks". In: *Physical review E* 69.2 (2004), p. 026113.

[87] NVIDIA. *NVIDIA Nemotron 3: Efficient and Open Intelligence*. White Paper. 2025. URL: https://arxiv.org/abs/2512.20856.

[88] Bruno A Olshausen and David J Field. "Emergence of simple-cell receptive field properties by learning a sparse code for natural images". In: *Nature* 381.6583 (1996), pp. 607–609.

[89] OpenAI. *gpt-oss-120b gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL]. URL: https://arxiv.org/abs/2508.10925.

[90] Rose Oughtred et al. “The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions”. In: *Protein Science* 30.1 (2021), pp. 187–200.

[91] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.

[92] Adam Paszke et al. “Pytorch: An imperative style, high-performance deep learning library”. In: *Advances in neural information processing systems* 32 (2019).

[93] Shishir G Patil et al. “Gorilla: Large language model connected with massive apis”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 126544–126565.

[94] Xinyang Pu et al. “ClassWise-SAM-adapter: Parameter-efficient fine-tuning adapts segment anything to SAR domain for semantic segmentation”. In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 18 (2025), pp. 4791–4804.

[95] Pranav Putta et al. “Agent q: Advanced reasoning and learning for autonomous ai agents”. In: *arXiv preprint arXiv:2408.07199* (2024).

[96] Yujia Qin et al. “Toolllm: Facilitating large language models to master 16000+ real-world apis”. In: *arXiv preprint arXiv:2307.16789* (2023).

[97] Alec Radford et al. “Language models are unsupervised multitask learners”. In: *OpenAI blog* 1.8 (2019), p. 9.

[98] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.

[99] Rajat Raina, Anand Madhavan, and Andrew Y Ng. "Large-scale deep unsupervised learning using graphics processors". In: *Proceedings of the 26th annual international conference on machine learning*. 2009, pp. 873–880.

[100] Jeff Rasley et al. "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters". In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 3505–3506.

[101] Nikhila Ravi et al. "Sam 2: Segment anything in images and videos". In: *arXiv preprint arXiv:2408.00714* (2024).

[102] Shaoqing Ren et al. "Faster R-CNN: Towards real-time object detection with region proposal networks". In: *IEEE transactions on pattern analysis and machine intelligence* 39.6 (2016), pp. 1137–1149.

[103] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *CoRR* abs/1505.04597 (2015). arXiv: 1505.04597. URL: http://arxiv.org/abs/1505.04597.

[104] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-net: Convolutional networks for biomedical image segmentation". In: *International Conference on Medical image computing and computer-assisted intervention*. Springer. 2015, pp. 234–241.

[105] Frank Rosenblatt. "The perceptron: a probabilistic model for information storage and organization in the brain." In: *Psychological review* 65.6 (1958), p. 386.

[106] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.

[107] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986),

pp. 533–536. DOI: 10.1038/323533a0. URL: https://www.nature.com/articles/323533a0.

[108] Olga Russakovsky et al. "ImageNet Large Scale Visual Recognition Challenge". In: *International Journal of Computer Vision (IJCV)* 115.3 (2015), pp. 211–252. DOI: 10.1007/s11263-015-0816-y.

[109] Shibani Santurkar et al. "How Does Batch Normalization Help Optimization?" In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018.

[110] Timo Schick et al. "Toolformer: Language models can teach themselves to use tools". In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.

[111] Yair Schiff et al. "Caduceus: Bi-directional equivariant long-range dna sequence modeling". In: *Proceedings of machine learning research* 235 (2024), p. 43632.

[112] John Schulman et al. "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[113] John Schulman et al. "Trust region policy optimization". In: *International conference on machine learning*. PMLR. 2015, pp. 1889–1897.

[114] Zhihong Shao et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models". In: *arXiv preprint arXiv:2402.03300* (2024).

[115] Noah Shinn et al. "Reflexion: Language agents with verbal reinforcement learning". In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.

[116] Jamie Shotton et al. "Textonboost: Joint appearance, shape and context modeling for multi-class object recognition and segmentation". In: *Computer Vision–ECCV 2006: 9th European Conference on Computer Vision, Graz, Austria, May 7-13, 2006. Proceedings, Part I 9*. Springer. 2006, pp. 1–15.

[117] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *arXiv preprint arXiv:1409.1556* (2014).

[118] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: 2601.03267 [cs.CL]. URL: https://arxiv.org/abs/2601.03267.

[119] Avi Singh et al. "Beyond human data: Scaling self-training for problem-solving with language models". In: *arXiv preprint arXiv:2312.06585* (2023).

[120] Nitish Srivastava et al. "Dropout: a simple way to prevent neural networks from overfitting". In: *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.

[121] Kyle A Sullivan et al. "Analyses of GWAS signal using GRIN identify additional genes contributing to suicidal behavior". In: *Communications Biology* 7.1 (2024), p. 1360.

[122] Kyle A. Sullivan et al. "MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research". In: *bioarXiv* (2024). DOI: 10.1101/2024.07.17.603821.

[123] Christian Szegedy et al. "Going deeper with convolutions". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1–9.

[124] Kimi Team et al. *Kimi K2: Open Agentic Intelligence*. 2026. arXiv: 2507.20534 [cs.LG]. URL: https://arxiv.org/abs/2507.20534.

[125] George Tsitsiridis et al. "CORUM: the comprehensive resource of mammalian protein complexes–2022". In: *Nucleic acids research* 51.D1 (2023), pp. D539–D545.

[126] L.C. Uzal et al. "Seed-per-pod estimation for plant breeding using deep learning". In: *Computers and Electronics in Agriculture* 150 (2018), pp. 196–204. ISSN: 0168-1699. DOI: https://doi.org/10.1016/j.compag.2018.04.024. URL: https://www.sciencedirect.com/science/article/pii/S016816991731582X.

[127] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[128] Petar Veličković et al. "Graph attention networks". In: *arXiv preprint arXiv:1710.10903* (2017).

[129] Pauli Virtanen et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[130] A. H. Vlot et al. "The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs". In: *PASC 2025*. 2025.

[131] Angelica M Walker et al. "Evaluating the performance of random forest and iterative random forest based methods when applied to gene expression data". In: *Computational and Structural Biotechnology Journal* 20 (2022), pp. 3372–3386.

[132] Stéfan van der Walt et al. "scikit-image: image processing in Python". In: *PeerJ* 2 (June 2014), e453. ISSN: 2167-8359. DOI: 10.7717/peerj.453. URL: https://doi.org/10.7717/peerj.453.

[133] Jiabo Wang and Zhiwu Zhang. "GAPIT version 3: boosting power and accuracy for genomic association and prediction". In: *Genomics, proteomics & bioinformatics* 19.4 (2021), pp. 629–640.

[134] Xuezhi Wang et al. "Self-consistency improves chain of thought reasoning in language models". In: *arXiv preprint arXiv:2203.11171* (2022).

[135] Zhizheng Wang et al. "GeneAgent: self-verification language agent for gene-set analysis using domain databases". In: *Nature Methods* 22.8 (2025), pp. 1677–1685.

[136] Jacob D Washburn et al. "Global genotype by environment prediction competition reveals that diverse modeling strategies can deliver satisfactory maize yield estimates". In: *Genetics* 229.2 (2025), iyae195.

[137] Jason Wei et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.

[138] Paul J Werbos. "Applications of advances in nonlinear sensitivity analysis". In: *System Modeling and Optimization: Proceedings of the 10th IFIP Conference New York City, USA, August 31–September 4, 1981*. Springer. 2005, pp. 762–770.

[139] Leandro von Werra et al. *TRL: Transformers Reinforcement Learning*. https://github.com/huggingface/trl. 2020.

[140] Chunlei Wu, Ian Macleod, and Andrew I Su. "BioGPS and MyGene.info: organizing online, gene-centric information". en. In: *Nucleic Acids Res.* 41.Database issue (Jan. 2013), pp. D561–5.

[141] Cuiling Wu et al. "A transformer-based genomic prediction method fused with knowledge-guided module". In: *Briefings in Bioinformatics* 25.1 (2023).

[142] Xide Xia and Brian Kulis. "W-net: A deep model for fully unsupervised image segmentation". In: *arXiv preprint arXiv:1711.08506* (2017).

[143] Enze Xie et al. "SegFormer: Simple and efficient design for semantic segmentation with transformers". In: *Advances in neural information processing systems* 34 (2021), pp. 12077–12090.

[144] Shunyu Yao et al. "React: Synergizing reasoning and acting in language models". In: *The eleventh international conference on learning representations*. 2022.

[145] Shunyu Yao et al. “Tree of thoughts: Deliberate problem solving with large language models”. In: *Advances in neural information processing systems* 36 (2023), pp. 11809–11822.

[146] Andy B. Yoo, Morris A. Jette, and Mark Grondona. “SLURM: Simple Linux Utility for Resource Management”. In: *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg, 2003, pp. 44–60. DOI: 10.1007/10968987_3.

[147] Zhihan Zhou et al. “Dnabert-2: Efficient foundation model and benchmark for multi-species genome”. In: *arXiv preprint arXiv:2306.15006* (2023).

Appendix A

Experimental Results

I Experiment 1

Results from Experiment 1 go here.

II Experiment 2

Results from Experiment 2 go here.

Vita

Vita goes here...